Algebraic Decomposition Theory for Transformer Length Generalization

Andy Yang
University of Notre Dame

Blerta Veseli
Universität des Saarlandes

Corentin Barloy
Ruhr-Universtät Bochum

Michaël Cadilhac
DePaul University

Andreas Krebs
Universität Tübingen

Charles Paperman
Université de Lille

Howard Straubing
Boston College

Michael Hahn
Universität des Saarlandes

Abstract

Transformer-based language models are known to sometimes generalize to sequences longer than seen during training, but we lack a precise characterization of which tasks admit length generalization. It is not even known which *regular languages* transformers length-generalize on – and these are a foundational class of languages which underlie state tracking, sequence formatting constraints, and agentic workflows. Our contributions are to establish the first complete characterization of which regular languages transformers length-generalize on and provide a decision algorithm running in polynomial time in DFA size. These results rely on an effective characterization of the regular languages in C-RASP, a recently-established formalism that expresses which languages transformers length-generalize on. This characterization is challenging because classical tools like Krohn-Rhodes decomposition theory for finite semigroups are insufficient for C-RASP. Firstly, the basic building blocks of Krohn-Rhodes theory – flip-flop and simple groups – are not expressible in C-RASP. Secondly, the basic building block of C-RASP (unbounded counting) is not expressible by the finite semigroups of Krohn-Rhodes theory. Thus, *length generalization on regular languages is controlled by an algebraic property that is invisible to classical finite decomposition theory.* We generalize classical decomposition theory from finite semigroups to the infinite additive group on the integers, allowing us to characterize C-RASP in terms of iterated wreath products of the integers and derive a provable polynomial-time decision algorithm for regular language membership. Experiments across a broad test suite of regular languages confirm that our theory captures transformers' length-generalization behavior more accurately than existing classifications.

1 Introduction

What state-tracking capabilities do transformer language models possess? To answer this question we study the *regular languages* on which transformers can *length-generalize*. First, regular languages give us a formal framework to describe the structure of state-tracking algorithms that transformers can implement. Second, probing for length-generalization gives us an empirical confirmation that the transformer learns an implementation of the underlying algorithm. Even though this is a fundamental question about an important architecture, we still lack a precise answer. For instance as shown in fig. 1, two structurally similar state-tracking tasks (recognizing $(ab + bbaa)^*$ and $(ab + aabb)^*$) may diverge sharply in terms of length-generalizability – and all existing theory fails to explain this discrepancy.

This line of inquiry has a deep history in machine learning. In fact Kleene (1956) invented regular expressions specifically for the purpose of analyzing the capabilities of McCulloch-

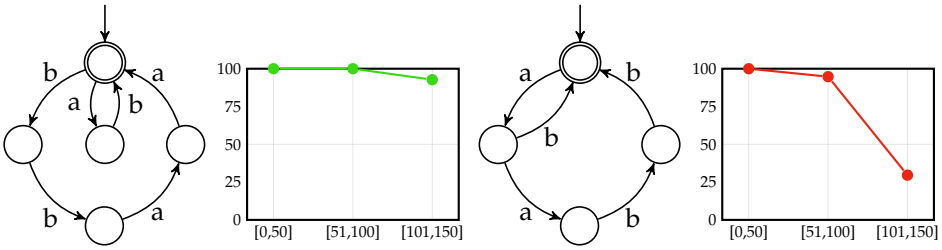


Figure 1: DFAs for $(ab + bbaa)^*$ and $(ab + aabb)^*$ and transformer length generalization on both. Transformers were trained on strings of length $[0, 50]$ and then tested on a held-out sets of strings from bins of length up to 150. Existing theory fails to explain why length-generalization differs between these simple regular languages with very similar structure.

Pitts neural networks. Regular language recognition is a fundamental task in the theory of computation that provides us a systematic method of determining the sequence-processing capabilities of a model (Sipser, 1996). The task is simple: given a sequence of inputs, each of which triggers a transition in a deterministic finite automaton (DFA), track the state of the machine after each transition. These finite state-tracking tasks arise concretely in the context of language models, which we sketch below.

Constrained decoding: A user may want a language model to generate text in a structured format, such as JSON or code, which can pose challenges (Schall & de Melo, 2025). Even as outputs grow in length, their structured formats often follow regular constraints (e.g. matching curly braces to a fixed depth in JSON).

Agentic workflows: AI agents in practice follow workflows consisting of compositional sequences of actions (Schluntz & Zhang, 2024). Keeping track of the state of the environment and deciding the next action to take can then explicitly be seen as simulating a DFA.

Natural language morphology: Well-formed sequences of morphemes in natural languages typically follow constraints that can be modeled by regular languages (Kaplan & Kay, 1994). In agglutinative languages, these morpheme sequences can grow long.

A previous line of work has characterized transformer *expressivity* on regular languages, which provides a theoretical foundation for our study. Liu et al. (2023b); Merrill & Sabharwal (2025) showed transformers with $\log(n)$ depth could express all regular languages, while constant depth could express all solvable regular languages. The containment of $\mathsf{poly}(n)$-precision transformers within TC^0 shows transformers cannot express non-solvable regular languages (assuming $\mathsf{TC}^0 \neq \mathsf{NC}^1$, as is common) (Merrill & Sabharwal, 2023; Chiang, 2025). Yang et al. (2024) and Li et al. (2024) showed that hard attention transformers expressed exactly the star-free regular languages. Li & Cotterell (2025) showed finite-precision transformers expressed exactly the $\mathcal{R}$-trivial regular languages.

In contrast, Bhattamishra et al. (2020); Huang et al. (2025) showed that transformers can length-generalize on languages both in and outside of each class discussed above, implying that *existing characterizations do not account for transformer length generalization on regular languages*. In particular, transformer length-generalization can be proven for all languages in C-RASP, and strong empirical evidence suggests failure of length-generalization outside of C-RASP (Huang et al., 2025). Prior to this work, though, there did not exist a complete characterization of the regular languages in C-RASP.

This is a deep and challenging theoretical question because classical tools for characterizing regular languages – namely the Krohn-Rhodes decomposition theory of Krohn & Rhodes (1965) – are insufficient for C-RASP. On one front, the building blocks of Krohn-Rhodes theory (flip-flop units and simple groups) are not expressible in C-RASP (Huang et al., 2025). On a second front, the basic building blocks of C-RASP (unbounded counting units) are not expressible by the finite semigroups of Krohn-Rhodes theory.

Our core innovation is to develop an analogous algebraic decomposition theory for transformers using the additive group on the integers. We effectively characterize the regular languages in C-RASP, and provide a polynomial-time algorithm that decides DFA membership in C-RASP. A coarser but simpler criterion is proven via a profinite equation. Finally, we validate empirically that regular language membership in C-RASP predicts transformer length-generalization better than any existing characterization.

2 Algebraic Preliminaries

Our characterization of the regular languages that languages transformers length-generalize on (i.e. those in C-RASP) builds upon the algebraic theory of formal languages (Pin, 2025).

2.1 Basics: Algebraic Theory of Formal Languages

We refer to appendix C for additional definitions. The essential ones are presented here.

Definition 1 (Monoid)**.** *A monoid $(M, \cdot, 1)$ is a set M with an associative binary operation and an identity element. We will just write M when the operation and identity are clear.*

A finite monoid M together with a homomorphism $\phi\colon \Sigma^* \to M$ behaves like a finite automaton: after reading string $w = w_1 \cdots w_n$, the automaton is in state $\phi(w_1) \cdots \phi(w_n)$. Two monoids will serve as basic units for us (the latter is the "flip-flop" alluded to above).

Definition 2. *The monoid U_1 is the set $\{0, 1\}$ where $0 \cdot 1 = 1 \cdot 0 = 0 \cdot 0 = 0$ and $1 \cdot 1 = 1$. The monoid U_2 is the set $\{1, a, b\}$ where $x \cdot a = a$, $x \cdot b = b$, and $1 \cdot x = x = x \cdot 1$ for any $x \in U_2$.*

As an example consider $L = \Sigma^* a \Sigma^*$ and the homomorphism $\phi\colon \Sigma^* \to U_1$ where $\phi(\sigma) = 0$ iff $\sigma = a$. Then for any $w \in \Sigma^*$ we have that $\phi(w) = 0 \in U_1$ iff in $w \in L$. In some sense, U_1 captures the structure of L (i.e. detect if a ever occurs in the string). More formally, we say that the *syntactic monoid* of L (denoted $M(L)$, and canonically associated to L) *divides* U_1.

Definition 3 (Division)**.** *A monoid M divides a monoid N iff there is a submonoid T of N and homomorphism $\phi\colon N \to M$ such that $M = \phi(T)$.*

2.2 Background: Classical Algebraic Decomposition Theory of Regular Languages

The classical algebraic operation for composing monoids is the *wreath product*.

Definition 4 (Classical Wreath Product)**.** *The wreath product $M \circ N$ of finite monoids $(M, +)$ and $(N, \cdot)$ is the monoid $M^N \times N$ with multiplication given by $(f_1, n_1)(f_2, n_2) = (f_1 + {}^{n_1}f_2, n_1 n_2)$, where the left action ${}^n f$ is given by ${}^n f(n') = f(n'n)$.*

The wreath product $M \circ N$ together with a homomorphism $\phi\colon \Sigma^* \to M \circ N$ behaves like the composition of a finite automaton and a finite transducer. Let ϕ_M and ϕ_N be such that $\phi(a) = (\phi_M(a), \phi_N(a))$ for all $a \in \Sigma$. After reading a prefix $w_1 \cdots w_t$, the automaton is in state $q_t = \phi_N(w_1) \cdots \phi_N(w_t)$, and the transducer is in state $\phi_M(w_1)(q_0) + \phi_M(w_2)(q_1) + \phi_M(w_3)(q_2) + \ldots + \phi_M(wt)(qt - 1)$. Wreath products are the backbone of the fundamental result in the decomposition theory of finite monoids, the Krohn-Rhodes Theorem:

Theorem 5 (Krohn-Rhodes Theorem (Krohn & Rhodes, 1965))**.** *Every finite monoid divides an iterated wreath product of U_2 and simple groups.*

As an recurring example, we show a decomposition of the bounded Dyck monoid.

Definition 6 (Bounded depth Dyck language)**.** *Define $\mathcal{D}_1 := (ab)^*$ and $\mathcal{D}_{k+1} := (a\mathcal{D}_k b)^*$.*

Proposition 7. *$M(\mathcal{D}_k)$ divides a wreath product of U_1 and $\mathbb{Z}_{k+3}$, the cyclic group of order $k + 3$.*

Proof. First consider the submonoid of $(U_1 \circ \mathbb{Z}_{k+3}) \times U_1$ generated by $((g, 0), 1)$, $((g, 1), 0)$, and $((g, k + 2), 0)$ where $g \in {U_1}^{\mathbb{Z}_{k+3}}$ given by $g(x) = 1 \iff x = k + 1$. It can be verified that $((g, 0), 1) \mapsto \epsilon$, $((g, 1), 0) \mapsto a$, and $((g, k + 2), 0) \mapsto b$ extends to a surjective homomorphism $h\colon U_1 \circ \mathbb{Z}_{k+3} \to M(\mathcal{D}_k) \times U_1$. Finally, using the fact that direct products divide wreath products, we conclude that $M(\mathcal{D}_k) \preceq (U_1 \circ \mathbb{Z}_{k+3}) \circ U_1$. □

Intuitively, computation in $\mathbb{Z}_{k+3}$ detects if the depth ever exceeds $k+1$, and this depth-violation is detected by U_1. Another U_1 detects non-emptiness of the string.

Unfortunately, classical decomposition theory is insufficient for handling transformer length-generalization. C-RASP can define non-regular languages, while also not being able to define U_2 (Huang et al., 2025). In short: transformers simultaneously succeed at length-generalizing on languages beyond the scope of Krohn-Rhodes theory (Bhattamishra et al., 2020), and fail to length generalize on the fundamental units in scope of Krohn-Rhodes theory (Liu et al., 2023a). This necessitates a new algebraic decomposition theory to handle C-RASP.

3 Algebraic Characterization of C-RASP

Huang et al. (2025) showed that transformer length-generalization can be guaranteed for all languages in C-RASP, and strong empirical evidence suggests failure of length-generalization outside of C-RASP (Jobanputra et al., 2025). Furthermore, a form of fixed-precision transformer is equivalent to C-RASP (Yang et al., 2025). In this section we develop an algebraic characterization of C-RASP, which crucially requires moving beyond Krohn-Rhodes theory to infinite monoids. We refer to Yang & Chiang (2024); Huang et al. (2025) for an exposition of C-RASP and provide a formal definition in appendix F.

3.1 Typed Monoids

Krebs (2008) developed a framework for using infinite monoids to recognize languages. We present a restriction of the aforementioned framework to the case of wreath products (a one-sided version of the block product used in previous work), which ultimately provides an exact algebraic characterization of C-RASP. The core issue here is that the wreath product of infinite monoids can generate uncountably many elements, which can be too powerful.

Proposition 8. *Consider the classic wreath product $\mathbb{Z}\circ\mathbb{Z}$. Then $M(L) \preceq \mathbb{Z}\circ\mathbb{Z}$ for every L.*

Proof. Without loss of generality let $\Sigma = \{0,1\}$. Consider the submonoid of $\mathbb{Z}\circ\mathbb{Z}$ generated by the image of Σ^* under the homomorphism $\sigma \mapsto (f_\sigma, 1)$ where $f_\sigma(x) = \sigma \cdot 2^x$. In essence, this creates a mapping $w \mapsto (f_w, |w|)$ where $f_w(0)$ outputs the integer value of the binary number w. Thus, $\mathbb{Z}\circ\mathbb{Z}$ can recognize arbitrary languages. □

This problem motivates the definition of *typed monoids*, which restricts the accepting sets to be collection of sets closed under union, intersection and complement.

Definition 9. *A typed monoid is a triple $(M, \mathfrak{T}_M, \mathcal{E}_M)$ where M is a finitely generated monoid, $\mathfrak{T}_M$ is a finite Boolean algebra over M, and $\mathcal{E}_M$ is a finite subset of M. Elements of $\mathfrak{T}_M$ are the* types *and elements of $\mathcal{E}_M$ are the* units. *A language L is recognized by $(S, \mathfrak{T}_M, \mathcal{E}_M)$ if there exists a homomorphism $h\colon \Sigma^* \to S$ such that $h(\Sigma) \subseteq \mathcal{E}_M$ and $L = h^{-1}(\mathfrak{M})$ for some $\mathfrak{M} \in \mathfrak{T}_M$.*

As an example, the language MAJORITY (there are more a's than b's) is recognized by the typed monoid $(\mathbb{Z}, \{(-\infty, 0], [1, \infty)\}, \{-1, 1\})$ via the type $[1, \infty)$ and the homomorphism $a \mapsto 1$ and $b \mapsto -1$. We will typically refer to this typed monoid as $\mathbb{Z}$. Now the *typed wreath* product follows the same intuition as the classical wreath product, except the computations are restricted so as not to have more distinguishing power than provided by the types.

Definition 10 (Typed Wreath Product)**.** *Let $(M, \mathfrak{T}_M, \mathcal{E}_M)$, $(N, \mathfrak{T}_N, \mathcal{E}_N)$ be two typed monoids, $C \subseteq N$ be a finite set. The typed wreath product*

$$(U, \mathfrak{T}_U, \mathcal{E}_U) = (M, \mathfrak{T}_M, \mathcal{E}_M) \circ_C (N, \mathfrak{T}_N, \mathcal{E}_N)$$

of $(M, \mathfrak{T}_M, \mathcal{E}_M)$ with $(N, \mathfrak{T}_N, \mathcal{E}_N)$ using constants C is defined such that

- *$\mathcal{E}_U$ consists of elements (f, n), where $t \in \mathcal{E}_N$, and $f : N \to \mathcal{E}_M$ is a type respecting function (see definition 30) with respect to $(N, \mathfrak{T}_N, \mathcal{E}_N)$ and C*
- *U is the submonoid of $M \circ N$ generated by $\mathcal{E}_U$*
- *$\mathfrak{T}_U$ consists of types $\mathfrak{U}_{\mathfrak{M},\mathfrak{N}} = \{(f, n) \mid f(1_M) \in \mathfrak{M}, n \in \mathfrak{N}\}$, where $\mathfrak{M} \in \mathfrak{T}_M$, $\mathfrak{N} \in \mathfrak{T}_N$*

3.2 Wreath Product Characterization

With the notion of typed monoids in hand, we can give an algebraic characterization of C-RASP. Let $\mathrm{wpc}(M, \mathfrak{T}_M, \mathcal{E}_M)$ denote the *wreath product closure* of a typed monoid, which closes iterated wreath prodcuts of this monoid under Boolean combinations and other basic operations. The precise definition will be found in appendix E.2. The typed wreath product closure turns out to be the precise algebraic "glue" that connects integer counting to C-RASP programs. The proof is given in appendix F.4.

Theorem 11. $L(\text{C-RASP}) = L(\mathrm{wpc}(\mathbb{Z}))$

4 Decomposition Theory for C-RASP

So far, we have characterized C-RASP in terms of iterated wreath products of $\mathbb{Z}$. We will turn this into an algebraic decision algorithm: given a regular language L, we determine if $M(L)$ divides a wreath product of $\mathbb{Z}$ (or not). A priori, such a question is a formidable problem, due to multiple challenges: $\mathbb{Z}$ is infinite, and we do not even know how many $\mathbb{Z}$ factors are needed. In fact, there even examples in the finite case where the problem ends up undecidable (Rhodes, 1999). Our second main result will be that, using a detailed understanding of wreath products of $\mathbb{Z}$, this problem is decidable.

In the depth-1 case, it is easy to check if $M(L)$ divides $\mathbb{Z}$ (this is true e.g. for the AND language, $L = 1^*$). How would we check if $M(L) \prec \mathbb{Z} \circ \mathbb{Z}$? If we could "divide" out the right factor to obtain an object "$M(L)/\mathbb{Z}$", we could check if "$M(L)/\mathbb{Z}$" $\prec \mathbb{Z}$. If this is not the case, we could "divide" by $\mathbb{Z}$ again, and iterate until we either reach division of $\mathbb{Z}$ (i.e., L is in C-RASP), or a fixed point (i.e., L is not in C-RASP). This is the basic idea of our decision procedure. There are three challenges at hand: First, understanding how to "divide" by a wreath product factor; second, making this computable even though $\mathbb{Z}$ is infinite; third, understanding how to identify the fixed point.

4.1 Categories

What do you get when you "divide" one monoid by another? For this first challenge, there is a well-understood technique using categories as algebraic structures (Tilson, 1987). We informally present the main idea, but defer the precise definitions to appendix G.

In group theory, this question has a simple answer. With a group homomorphism between groups $\phi\colon G \to H$, we can "divide" G by $\ker\phi$ with a "quotient" of H, such that $G \preceq (\ker\phi)\circ H$. However, since C-RASP contains no non-trivial groups, this is of no use for us.

For monoid homomorphisms $\phi\colon M \to N$, we may not be able to "divide" M by $\ker\phi$, as monoids' lack of inverses obstructs such a clean division. What is the divisor in the division $\phi\colon M(\mathcal{D}_1) \to U_2$. Intuitively, the wreath product $M(\mathcal{D}_1) \preceq N \circ U_2$ should form a structure that contains elements $(\text{last symbol} = a)$ and $(\text{last symbol} = b)$. However, composition in this structure will be highly restricted – e.g. $(\text{this symbol} = b)(\text{last symbol} = a)$ is an illegal composition. It turns out the cleanest way to handle this is by lifting monoids and homomorphisms to higher order structures – categories and relational morphisms[1].

A category X comprises of a set of *objects* $\mathrm{Obj}(X)$ and *hom-sets* $X(x_1, x_2)$, which are collections of *arrows* $\alpha\colon x_1 \to x_2$ for each pair of objects $x_1, x_2 \in X$. Arrows can compose associatively, and each object has an identity arrow. The objects of a category can be viewed as "local monoids", within which all arrows allowed to compose with each other. A relational morphism of monoids $\phi\colon M \triangleleft N$ is a relation where $\phi(m) \neq \emptyset$ and $\phi(m_1)\phi(m_2) \subseteq \phi(m_1 m_2)$. Intuitively, these relations are generated by choosing any function $M \to N$ and closing it under multiplication in M to obtain a homomorphism-like mapping.

Now for any relation morphism of monoids $\phi\colon M \triangleleft N$, the correct notion of divisor turns out to be the *derived category* D_ϕ. Here, $\mathrm{Obj}(D_\phi) = \phi(M)$ and $D_\phi(n_1, n_2) = \{n_1 \to_{(m,n)} \mid n \in$

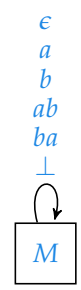
[1] One *could* model this in terms of monoids like Tilson (1974), but using categories creates a much cleaner model, and is in some sense the "correct" way to do it.

$\phi(m), n_1 n = n_2\}$. Arrows compose via the rule $n_0 \to_{(m_1,n_1)} n_0 n_1 \to_{(m_2,n_2)} = n_0 \to_{(m_1 m_2, n_1, n_2)}$, and we merge arrows that have the same behavior under composition.

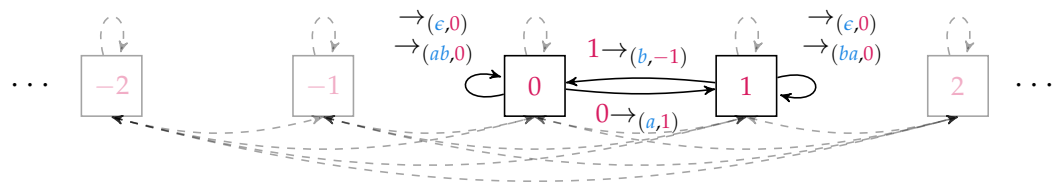
The *Derived Category Theorem* states that given a relational morphism $\phi\colon M \triangleleft N$, and a division $D_\phi \preceq V$, we obtain a division $M \preceq V \circ N$. Thus if $\phi : M \triangleleft \mathbb{Z}$, then D_ϕ formalizes the object "$M/\mathbb{Z}$" alluded to in the previous section.

4.2 Decomposition of $\mathcal{D}_1$ into wreath products of $\mathbb{Z}$

We explain our decision procedure by walking through the division of the syntactic monoid of $\mathcal{D}_1 = (ab)^*$ into an iterated wreath product of $\mathbb{Z}$, via carefully selected relational morphisms into $\mathbb{Z}$. We will visualize objects of a category as rectangles (to prevent confusion with automata) and arrows as labels on edges between squares. Colors indicate the monoid each element comes from. We can view $M(\mathcal{D}_1)$ as a category with a single object and arrows corresponding to monoid elements. We take a relational morphism into $\phi_1\colon M(\mathcal{D}_1) \triangleleft \mathbb{Z}$ where $\phi_1(\perp) = \mathbb{Z}$, $\phi_1(a) = \{1\}$, $\phi_1(b) = \{-1\}$, $\phi_1(\epsilon) = \phi_1(ab) = \phi_1(ba) = \{0\}$.

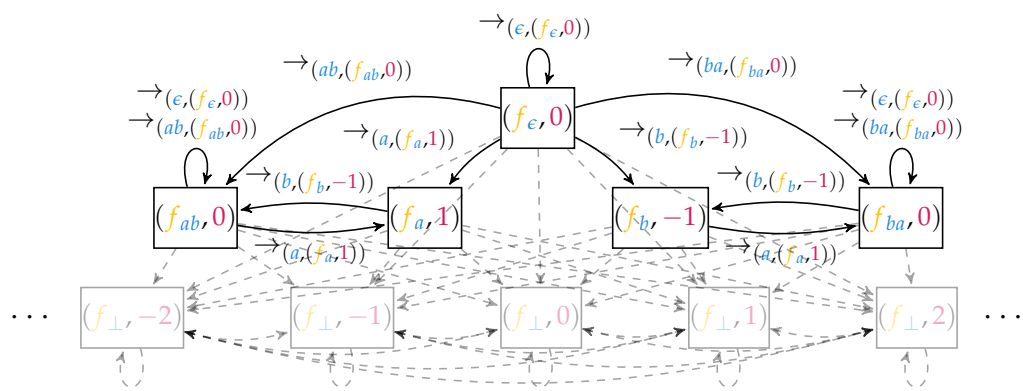


In the derived category D_{ϕ_1} there are infinitely many objects, corresponding to each integer. We omit the starting object of each arrow (as it is clear by the diagram), combine isomorphic arrows, and let low-opacity arrows and objects denote transitions to $\perp$ elements.



We have not yet arrived at a division $M(\mathcal{D}_1) \preceq \mathbb{Z}$ because D_{ϕ_1} has hom-sets containing multiple arrows (Tilson, 1987, Lemma 3.1, Lemma 4.1). To proceed, we take another relational morphism $\psi_1\colon D_{\phi_1} \triangleleft \mathbb{Z}$ where $\psi_1(\to) = \{0\}$ if $\to \in \left\{0\to_{(\epsilon,0)}, 0\to_{(ab,0)}, 1\to_{(ba,0)}, 0\to_{(a,1)}, 1\to_{(b,-1)}, 1\to_{(\epsilon,0)}\right\}$ and $\psi_1(\to) = [1, \infty)$ otherwise.

From ϕ_1, ψ_1 we define a relational morphism $\phi_2\colon M(\mathcal{D}_1) \triangleleft \mathbb{Z} \circ \mathbb{Z}$ where we let $f_{(m,y)}(x) = \psi_1\left(x \to_{(m,y)}\right)$ in $\phi_2(m) = \left\{\left(f_{(m,y)}, y\right) \mid y \in \phi_1(m)\right\}$. We visualize the derived category D_{ϕ_2} below (writing (f_m, y) as shorthand for $\phi_2(m)$ whenever it is unique).



We still do not have a division because there are still two arrows in a single homset of D_{ϕ_2} (e.g. $\phi_2(\epsilon)$ and $\phi_2(ab)$ act the same on $(f_{ab}, 0)$). To handle this, we define a final

relational morphism $\phi_3 \colon M(\mathcal{D}_1) \triangleleft \mathbb{Z} \circ \mathbb{Z} \circ \mathbb{Z}$ such that $\phi_3(m) = (g, \phi(m))$ where $g(x) = 0$ iff $x = (f_{(\epsilon,0)}, 0)$. We ultimately obtain the division. $\phi_3 \colon M(\mathcal{D}_1) \preceq \mathbb{Z} \circ \mathbb{Z} \circ \mathbb{Z}$. In this example, we chose the correct relational morphisms into $\mathbb{Z}$ so as to obtain a division, but in principle there are infinitely many choices of morphisms. Guiding the choice of relational morphisms to reach a termination is a major challenge we address in our decision procedure.

4.3 Algebraic Decision Procedure

We now address the second challenge of maintaining computability despite the existence of infinitely many relational morphisms into $\mathbb{Z}$. First, we divide $M(L)$ into equivalence classes via the $\mathcal{R}$ relation, a classical relation in algebraic automata theory which characterizes which elements are reachable by other elements via right-multiplication (Colcombet, 2011). We construct a sequence of iterated relational morphism into $\mathbb{Z}$ by iterating over the $\mathcal{R}$-classes in order. At step $i+1$, we assume that a relational morphism ϕ_i covers all $\mathcal{R}$-classes up to R_i, and extend to a relational morphism ϕ_{i+1} that covers R_{i+1}. This is done by iteratively computing nontrivial relational morphisms whose values are bounded on R_i. This process terminates, because the set of such bounded relational morphisms forms a finite-dimensional $\mathbb{Z}$-module (an integer analogue of vector spaces), and every step finds a morphism linearly independent of the previous ones.

Finally we address the termination conditions by proving completeness of the algorithm. If the algorithm terminates with success, we have constructed a division from $M(L)$ into a wreath product of $\mathbb{Z}$ (and thus C-RASP by theorem 11). Otherwise, if the algorithm terminates with failure, we show no such division exists. In this case, assume for sake of contradiction that $M(L)$ divides a T-fold iterated wreath product of $\mathbb{Z}$ via a relational morphism $\psi \colon M(L) \preceq \mathbb{Z} \circ \cdots \circ \mathbb{Z}$ (where T is minimal). The rightmost component defines a relational morphism $\omega : M \triangleleft \mathbb{Z}$. Either ω is bounded, and it would have been chosen as part of the decomposition had it provided any useful information, or ω is unbounded which renders long strings indistinguishable by types in $\mathbb{Z}$ since values may run to infinity. In either case, we can remove the rightmost component of ψ, and obtain a division from M into a $(T-1)$-fold iterated wreath product of $\mathbb{Z}$, contradicting the minimality of T.

To formalize the above proof, we choose relational morphisms to $\mathbb{Z}$ which truncate values beyond a threshold k (which results in relational morphisms to $\mathcal{D}_k$). This allows us to formalize the proof using finite category theory (avoiding the use of types on the category side). A formal writeup can be found in appendix H.2.

4.4 Necessary (but not Sufficient) Condition via Equations

In this section use profinite equations to give an alternative characterization of the regular languages in C-RASP. These equations are a technique drawing ideas from topology to derive elegant and decidable characterizations for classes of monoids. We defer a precise definition to Pin (2009).

Definition 12. *Define $\mathbf{R}^\omega$ as an equation and the corresponding variety of monoids*[2] *as*

$$\mathbf{R}^\omega : (xy^\omega)^\omega x = (xy^\omega)^\omega.$$

For our purposes, a finite monoid M satisfies $\mathbf{R}^\omega$ if for any $m_1, m_2 \in M$ we have that $(m_1 m_2^\omega)^\omega = (m_1 m_2^\omega)^\omega m_1$, where m^ω denotes m^k such that $m^k = m^{2k}$ (i.e. the unique idempotent generated by m). We give an exact characterization of the monoids in $\mathbf{R}^\omega$, the proof of which is in appendix I.

Theorem 13. *$M \in \mathbf{R}^\omega$ iff M is aperiodic and every $\mathcal{R}$-class of M contains at most one idempotent.*

While both the main decision procedure and $\mathbf{R}^\omega$ require iterating over the $\mathcal{R}$-classes of the monoid. the former needs to iteratively compute relational morphisms, while the latter only needs to count idempotents. This gives a much simpler criterion that is *necessary* for membership in C-RASP, though we can show it is *not sufficient*.

[2] $\mathbf{R}^\omega$ alludes to the equation for $\mathcal{R}$-trivial monoids, $(xy)^\omega = (xy)^\omega x$ (Almeida & Azevedo, 1989)

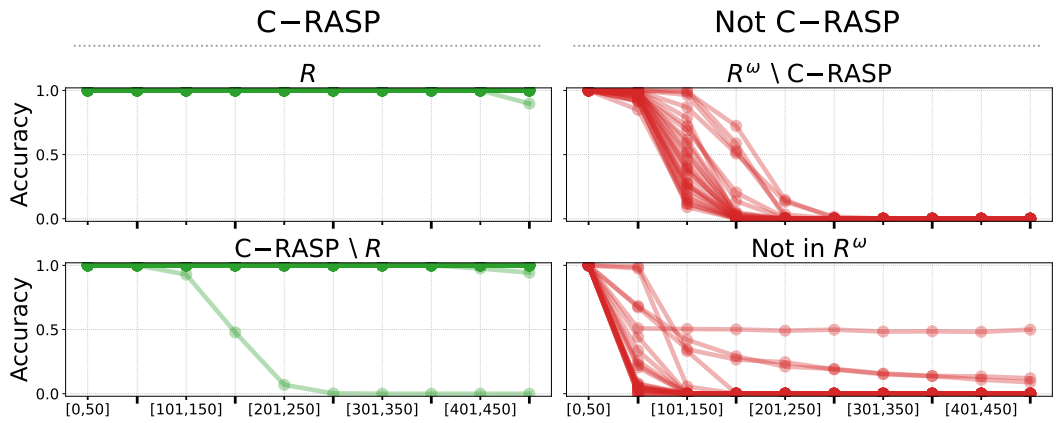


Figure 2: **Length generalization on regular languages.** Models are trained on strings of lengths in $[l_{\min}, 50]$, and evaluated on bins $[l_{\min}, 50]$ (in-distribution) up to $[451, 500]$ in bins of width 50. Each curve corresponds to a language. Green curves denote languages in C-RASP, while red curves denote languages not in C-RASP. Languages in C-RASP maintain near-perfect accuracy well beyond the training range, whereas languages outside C-RASP exhibit rapid degradation, typically failing shortly after. Each panel includes all languages in the corresponding class from Table 1 in appendix B.4.

4.5 Algebraic Characterization

The decision procedure results an an exact algebraic characterization of the regular languages in C-RASP – namely, they are wreath products of bounded-depth Dyck languages. We thus derive a small hierarchy within the classes of finite monoids surrounding C-RASP. Informally, **R** is the class studied by Li & Cotterell (2025), **A** is the class studied by Yang et al. (2024), and **REG** is all regular languages (formal definitions are in appendix C).

Theorem 14. $\text{C-RASP} \cap \mathbf{REG} = \text{wpc}(\mathbf{Dy})$. *Hence* $\mathbf{R} \subsetneq \text{C-RASP} \cap \mathbf{REG} \subsetneq \mathbf{R}^{\omega} \subsetneq \mathbf{A} \subsetneq \mathbf{REG}$

A proof is given in appendix I. Furthermore, the decision procedure runs in polynomial time in the size of the monoid. This allows us to efficiently decide whether or not we expect a transformer to length-generalize on any given regular language. A proof is in appendix H.2.

Theorem 15. *Membership of* $M \in \text{C-RASP} \cap \mathbf{REG}$ *is decidable in* $O(\text{poly}(|M|))$ *time.*

5 Experiments

We empirically evaluate whether our characterization of regular languages in C-RASP predicts transformer length-generalization. In particular, we will examine the different levels of the hierarchy shown in theorem 14 to determine the efficacy of each class in predicting length-generalization by transformers.

5.1 Languages

We construct a diverse suite of 130 regular languages, including both representative examples from prior work (Li & Cotterell, 2025; Huang et al., 2025) and systematically generated ones. We provide the full table of languages in Table 1, appendix B). To generate languages within the classes discussed in theorem 14, we sample regular expressions using a probabilistic context-free grammar (PCFG) for which we tune the probabilities to obtain diverse languages within those classes. For each language, we determine membership in C-RASP using our polynomial-time decision procedure. We implement an automata-based version of the algebraic procedure, which we explain in appendix J.

5.2 Experimental Setup

Task Definition. The task is state prediction, i.e. tracking automaton states over prefixes. Let $w = a_1 \dots a_n$, and denote by $w_{1:i} = a_1 \dots a_i$ the prefix of length i. Each symbol a_i triggers a transition, and the model must predict the sequence of DFA states $q_1, \dots, q_n$, where q_i is the state reached after processing $w_{1:i}$.

Length Generalization. We train models on words with lengths within the range $[l_{min}, 50]$ and evaluate on bins $[l_{min}, 50], [51, 100], [101, 150] \dots [451, 500]$. We define successful length generalization as maintaining near in-distribution performance at lengths beyond $2\times$ the maximum training length 50.

Architectural Constraints. To align empirical results with our theoretical setup, state prediction should depend on the evolving prefix rather than local information. We therefore introduce two constraints: first, we remove positional information by replacing positional embeddings with the zero function (NoPE) forcing the model to rely on the sequential order of tokens provided by the causal attention mask. Second, while NoPE removes absolute positional information, the model still has direct access to the most recent input symbol a_i when predicting the target state q_i, which provides a positional signal through alignment with the prediction target. This can enable shortcuts in which the model predicts the state based solely on the final symbol, rather than the full prefix. To prevent this, we insert a separator token & between symbols (see below) and require the model to predict the state q_i only at these separator positions. Consequently, access to the most recent symbol is also mediated through the attention mechanism. Word symbols receive placeholder targets # and are excluded from the cross-entropy loss. Each target sequence begins with the initial DFA state, predicted at the first separator following <bos>.

Input	<BOS>	&	a	&	b	&	a	&	b	&	a	&	b	&	<EOS>
Positional Encoding	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Target	#	1	#	3	#	1	#	3	#	1	#	3	#	1	#

5.3 Training and Evaluation.

We adopt a hyperparameter search procedure similar to Huang et al. (2025), but select best configuration based on performance up to length 150 to account for our longer length generalization setting. For the final results, we train 5 models with different random initializations. We report the performance of the best seed in Figure 2 with average performance across all seeds provided in B.

5.4 Results

Figure 2 shows that C-RASP membership is a strong predictor of transformer length-generalization. Languages in C-RASP generalize reliably to substantially longer lengths, while languages outside C-RASP fail to do so, with accuracy rapidly collapsing beyond the training length. This demonstrates that across all evaluated languages C-RASP provides an accurate characterization of length generalization.

6 Conclusion

We have precisely characterized the regular languages on which transformers length-generalize using a decision procedure for regular language membership in C-RASP (which runs in polynomial-time). Experiments on a range of languages in and outside of C-RASP demonstrate that the theory accurately predicts when transformers do and do not length-generalize. The decision procedure is based on a novel algebraic decomposition theory for C-RASP that is distinguished from the classical Krohn-Rhodes theory due to the inclusion of infinite monoids and the omission of the aperiodic flip-flop unit U_2. A simpler criterion which is necessary (but not sufficient) for membership in C-RASP is also given in the form

of a profinite equation. Our results provide a deeper understanding of the state-tracking capabilities of transformers on the machine learning front, as well as a deeper understanding of counting, on the algebraic front.

References

Jorge Almeida and Assis Azevedo. The join of the pseudovarieties of r-trivial and l-trivial monoids. *Journal of Pure and Applied Algebra*, 60(2):129–137, 1989. ISSN 0022-4049. doi: https://doi.org/10.1016/0022-4049(89)90125-4. URL https://www.sciencedirect.com/science/article/pii/0022404989901254.

Eric Alsmann, Lowejatan Noori, and Martin Lange. On the expressiveness of state space models via temporal logics. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=Vg511oJScS.

Christoph Behle, Andreas Krebs, and Stephanie Reifferscheid. Typed monoids – an eilenberg-like theorem for non regular languages. In Franz Winkler (ed.), *Algebraic Informatics*, pp. 97–114, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-21493-6. doi: https://doi.org/10.1007/978-3-642-21493-6_6.

Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. On the Ability and Limitations of Transformers to Recognize Formal Languages. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7096–7116, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.576. URL https://aclanthology.org/2020.emnlp-main.576/.

J.A. Brzozowski and Faith E. Fich. Languages of r-trivial monoids. *Journal of Computer and System Sciences*, 20(1):32–49, 1980. ISSN 0022-0000. doi: https://doi.org/10.1016/0022-0000(80)90003-3.

David Chiang. Transformers in uniform TC^0. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=ZA7D4nQuQF.

Thomas Colcombet. Green's Relations and Their Use in Automata Theory. In *Language and Automata Theory and Applications*, volume 6638 of *Lecture Notes in Computer Science*, pp. 1–21, Tarragone, Spain, May 2011. Springer. doi: 10.1007/978-3-642-21254-3_1. URL https://hal.science/hal-00638599.

Xinting Huang, Andy Yang, Satwik Bhattamishra, Yash Sarrof, Andreas Krebs, Hattie Zhou, Preetum Nakkiran, and Michael Hahn. A formal framework for understanding length generalization in transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=U49N5V51rU.

Mayank Jobanputra, Yana Veitsman, Yash Sarrof, Aleksandra Bakalova, Vera Demberg, Ellie Pavlick, and Michael Hahn. Born a transformer – always a transformer? on the effect of pretraining on architectural abilities. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=Huw15LqglI.

Ronald M. Kaplan and Martin Kay. Regular models of phonological rule systems. *Computational Linguistics*, 20(3):331–378, 1994. URL https://aclanthology.org/J94-3001/.

S. C. Kleene. *Representation of Events in Nerve Nets and Finite Automata*, pp. 3–42. Princeton University Press, Princeton, 1956. ISBN 9781400882618. doi: doi:10.1515/9781400882618-002. URL https://doi.org/10.1515/9781400882618-002.

Andreas Krebs. *Typed Semigroups, Majority Logic, and Threshold Circuits*. PhD thesis, Universität Tübingen, 2008. URL https://nbn-resolving.org/urn:nbn:de:bsz:21-opus-36244. URN: urn:nbn:de:bsz:21-opus-36244.

Kenneth Krohn and John Rhodes. Algebraic theory of machines. I. Prime decomposition theorem for finite semigroups and machines. *Transactions of the American Mathematical Society*, 116:450–464, 1965. doi: 10.1090/S0002-9947-1965-0188316-1. URL https://doi.org/10.1090/S0002-9947-1965-0188316-1.

Jiaoda Li and Ryan Cotterell. Characterizing the expressivity of fixed-precision transformer language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=29LwAgLFpj.

Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=3EWTEy9MTM.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Exposing attention glitches with flip-flop language modeling. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023a. URL https://openreview.net/forum?id=VzmpXQAn6E.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *The Eleventh International Conference on Learning Representations*, 2023b. URL https://openreview.net/forum?id=De4FYqjFueZ.

William Merrill and Ashish Sabharwal. A logic for expressing log-precision transformers. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 52453–52463. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a48e5877c7bf86a513950ab23b360498-Paper-Conference.pdf.

William Merrill and Ashish Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=5pHfYe10iX.

Jean-Eric Pin. Profinite Methods in Automata Theory. In Susanne Albers and Jean-Yves Marion (eds.), *26th International Symposium on Theoretical Aspects of Computer Science*, volume 3 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 31–50, Dagstuhl, Germany, 2009. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-939897-09-5. doi: 10.4230/LIPIcs.STACS.2009.1856. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2009.1856.

Jean-Éric Pin. Mathematical foundations of automata theory. Lecture notes, MPRI, IRIF, CNRS and Université de Paris. Available at https://www.irif.fr/~jep/PDF/MPRI/MPRI.pdf, 2025.

John Rhodes. Undecidability, automata, and pseudovarities of finite semigroups. *International Journal of Algebra and Computation*, 9(3):455–474, 1999. doi: 10.1142/S0218196799000278. URL https://doi.org/10.1142/S0218196799000278.

Yash Sarrof, Yana Veitsman, and Michael Hahn. The expressive capacity of state space models: A formal language perspective. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=eV5YIrJPdy.

Maximilian Schall and Gerard de Melo. The hidden cost of structure: How constrained decoding affects language model performance. In Galia Angelova, Maria Kunilovskaya, Marie Escribe, and Ruslan Mitkov (eds.), *Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing - Natural Language Processing in the Generative AI Era*, pp. 1074–1084, Varna, Bulgaria, September 2025. INCOMA Ltd., Shoumen, Bulgaria. URL https://aclanthology.org/2025.ranlp-1.124/.

Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, December 2024. Accessed: 2026-03-23.

Michael Sipser. *Introduction to the Theory of Computation*. International Thomson Publishing, 1st edition, 1996. ISBN 053494728X. URL https://dl.acm.org/doi/10.5555/524279.

Price Stiffler Jr. Extension of the fundamental theorem of finite semigroups. *Advances in Mathematics*, 11(2):159–209, 1973. doi: https://doi.org/10.1016/0001-8708(73)90007-8.

Denis Thérien and Thomas Wilke. Temporal logic and semidirect products: An effective characterization of the until hierarchy. *SIAM Journal on Computing*, 31(3):777–798, 2001. doi: 10.1137/S0097539797322772. URL https://doi.org/10.1137/S0097539797322772.

B. Tilson. On the complexity of finite semigroups. *Journal of Pure and Applied Algebra*, 5(2): 187–208, 1974. ISSN 0022-4049. doi: https://doi.org/10.1016/0022-4049(74)90046-2. URL https://doi.org/10.1016/0022-4049(74)90046-2.

Bret Tilson. Categories as algebra: An essential ingredient in the theory of monoids. *Journal of Pure and Applied Algebra*, 48(1):83–198, 1987. ISSN 0022-4049. doi: https://doi.org/10.1016/0022-4049(87)90108-3. URL https://doi.org/10.1016/0022-4049(87)90108-3.

Sam van der Poel, Dakotah Lambert, Kalina Kostyszyn, Tiantian Gao, Rahul Verma, Derek Andersen, Joanne Chau, Emily Peterson, Cody St. Clair, Paul Fodor, Chihiro Shibata, and Jeffrey Heinz. Mlregtest: A benchmark for the machine learning of regular languages. *Journal of Machine Learning Research*, 25(283):1–45, 2024. URL http://jmlr.org/papers/v25/23-0518.html.

Andy Yang and David Chiang. Counting like transformers: Compiling temporal counting logic into softmax transformers. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=FmhPg4UJ9K.

Andy Yang, David Chiang, and Dana Angluin. Masked hard-attention transformers recognize exactly the star-free languages. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=FBMsBdH0yz.

Andy Yang, Michaël Cadilhac, and David Chiang. Knee-deep in c-RASP: A transformer depth hierarchy. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=jPduiyxyfw.

Contents

1 Introduction **1**

2 Algebraic Preliminaries **3**

2.1 Basics: Algebraic Theory of Formal Languages 3

2.2 Background: Classical Algebraic Decomposition Theory of Regular Languages 3

3 Algebraic Characterization of C-RASP **4**

3.1 Typed Monoids 4

3.2 Wreath Product Characterization 5

4 Decomposition Theory for C-RASP **5**

4.1 Categories 5

4.2 Decomposition of $\mathcal{D}_1$ into wreath products of $\mathbb{Z}$ 6

4.3 Algebraic Decision Procedure 7

4.4 Necessary (but not Sufficient) Condition via Equations 7

4.5 Algebraic Characterization 8

5 Experiments **8**

5.1 Languages 8

5.2 Experimental Setup 9

5.3 Training and Evaluation. 9

5.4 Results 9

6 Conclusion **9**

A FAQ **15**

B Experimental Results **17**

B.1 Longer Training Length 17

B.2 Length Generalization Across Seeds 17

B.3 Increased Train Data Size 18

B.4 Languages 19

C Algebraic Preliminaries **19**

D Causally-masked programs **21**

E Typed Monoids **23**

E.1 Definitions 23

E.2 Proof of Typed Wreath Product Principle 26

F C-RASP **27**
F.1 Definitions . . . 27
F.2 C-RASP as a variety of Languages . . . 28
F.3 Existing Characterizations . . . 29
F.4 Algebraic Characterization of C-RASP . . . 30

G Derived Categories **30**

H Algebraic Decision Procedure **31**
H.1 Relevant Lemmas . . . 31
H.2 Decidability Proof (via Finite Derived Categories) . . . 31

I Algebraic Characterization (Necessary but not Sufficient Criterion) **38**

J Automata proof of C-RASP ∩ REG **39**

K Implementation **43**

A FAQ

1. *Q: How does the work relate and compare to Liu et al. (2023b)?*

   Liu et al. (2023b) primarily looked at expressivity, not length generalization. Indeed, their experiments also confirmed that transformers may not length-generalize on the particular regular languages that they were able to express and learn to a fixed length.

   Our results may also apply to the expressive power of transformers under a particular fixed-precision assumption, because C-RASP was shown to be equivalent to these transformers by Yang et al. (2025).

2. *Q: How about other architectures? Like log-depth transformers, state-space models, ...?*

   It is already known that log-depth architectures can simulate arbitrary automata, so a characterization of the regular langauges they can express is not needed (Liu et al., 2023b; Merrill & Sabharwal, 2025). As for length-generalization, we do not presently know what regular langauges these architectures length-generalize on.

   Analyzing architectures with limited recurrence, like state-space models, would be an interesting future question. Under varying assumptions, state-space models are able to simulate flip-flops and counting (Sarrof et al., 2024; Alsmann et al., 2026). The techniques we have developed to handle counting could be extended to handle these cases as well.

3. *Q: Why not evaluate LLMs?*

   We're more interested in the architecture itself. LLM abilities depend on prompt format and are strongly impacted by what's in the training data. There is work suggesting that the capabilities of LLMs are ultimately bounded by C-RASP (Jobanputra et al., 2025), though a precise investigation in the case of regular language length-generalziation is out of the scope of this work.

4. *Q: How does* C-RASP *compare to other classes, such as subregular classes, circuit classes, and the dot depth hierarchy? Could they on their own already predict transformer length generalization?*

   C-RASP is distinct from known classes, and much more successful at predicting length generalization than those are. In fact, all existing classes do not predict length-generalization on transformers.

- Star-Free and Non-Star-Free: C-RASP defines a strict subset of the star-free languages. As we show, even $\mathbf{R}^\omega$ only covers star-free languages (and in fact it is a strict subset).
- The circuit classes most relevant to transformers are AC^0 and TC^0; neither of them captures C-RASP or transformer length generalization well. C-RASP is a strict subset of TC^0, as was shown in Huang et al. (2025). This extends to regular languages: E.g., the PARITY language $(b^*ab^*ab^*)$ (checking if the number of a's is even) is in TC^0 but not in C-RASP. In general, C-RASP is incomparable with AC^0. Interestingly, on the level of regular languages, C-RASP is a strict subset of AC^0: C-RASP only includes star-free languages (and all of those are in AC^0), but, for instance, $\{a,b\}^*b$ is in AC^0 but not in C-RASP.
- Dot-depth hierarchy. C-RASP can express every bounded-depth dyck langauges, and thus can touch every single level of the dot-depth hierarchy, while not covering the entire hierarchy.

5. *Q: What about the role of positional encodings?*
   This is an important question for future work that would result in a different characterization than the one we have derived here. Huang et al. (2025) showed that C-RASP[periodic, local] characterizes the languages which transformers with APE can length-generalize on. Obtaining a decision procedure on the algebraic side would require additional techniques from the ones used in this paper.
6. *Q: Why did you not use MLRegTest for your experiments (van der Poel et al., 2024)?*
   Most of the languages in the dataset contain *strictly local* patterns, which are not expressible in C-RASP without positional encodings (Huang et al., 2025). Future work extending the characterization of C-RASP to handle this should use this benchmark.
7. *Q: The experiments deliberately prevent residual-stream access to the last token in state prediction. Why? Could the theory handle the case where one actually wants this access?*
   We argue that state tracking should be robust to the insertion of "do-nothing" actions not changing the state, or other unrelated extra material. In formal language theory, such actions are referred to as *neutral symbols*. Invariance under the inclusion of such neutral symbols is a natural property of characterizations in terms of syntactic monoids, as we have done here. An example where this plays a role is $\{a,b\}^*b$ (where the state – *is the last symbol seen a b?* – can be easily tracked based on the last symbol), which has the same syntactic monoid as the language $\{a,b,e\}^*be^*$ resulting from adding a neutral symbol e, which is difficult for Transformer length generalization (Liu et al., 2023a; Huang et al., 2025). Neither is definable in C-RASP.
8. *Q: Why do your empirical results differ from that of Li & Cotterell (2025) and Huang et al. (2025) who both test length generalization for regular languages in* C-RASP*?*
   We would like to address the empirical and theoretical results of Huang et al. (2025) and Li & Cotterell (2025) in light of our findings. Li & Cotterell (2025) claimed that transformers consistently fail to length-generalize on languages outside of $\mathbf{R}$, providing evidence when trained on length N and tested on length $12N$. While Huang et al. (2025) showed length-generalization in languages in C-RASP $\setminus$ $\mathbf{R}$, they only trained on length N and tested up to $3N$, so the results may be incomparable. In this work we specifically probed languages in C-RASP $\cap$ $\mathbf{REG}$ $\setminus$ $\mathbf{R}$ and found length-generalization from length N to $10N$, providing a body of evidence against the claim of Li & Cotterell (2025).

   We suspect the difference in our empirical observations can be attributed to the lack of languages tested by Li & Cotterell (2025) in C-RASP $\cap$ $\mathbf{REG}$ $\setminus$ $\mathbf{R}$ – only 3 such languages were tested. Indeed, prior to this work there existed no sizeable dataset of languages in C-RASP $\cap$ $\mathbf{REG}$ $\setminus$ $\mathbf{R}$, as there was no computable membership criterion for C-RASP $\cap$ $\mathbf{REG}$.

   An additional difference is that Li & Cotterell (2025) use a language *classification* task where the model is trained on both positive and negative examples. The experiments done by Bhattamishra et al. (2020); Huang et al. (2025); Yang et al. (2025), and this work on C-RASP all use a *next-token prediction* setup.

B Experimental Results

In the following section we provide additional experimental results and supporting material to section 5. In particular, we include (1) experiments with extended training ranges in B.1, (2) results aggregated across multiple random seeds (model initializations) in B.2, (3) effect of train data size in B.3 and (4) the complete list of regular languages used in our evaluation in B.4.

B.1 Longer Training Length

In Figure 3 we further evaluate length generalization on a suite of 50 regular languages with higher nesting depth compared to those analyzed in Table 1 (see Table 2 for details). These experiments utilize the same hyperparameter search and training dynamics described in the main paper, but with extended training ranges up to length 200. This allows us to assess whether the observed generalization behavior persists under longer training sequences and increased language complexity.

As shown in Figure 5, the results show similar trends from the shorter training range: models trained on languages within C-RASP (green) maintain near-perfect accuracy even when evaluated on lengths more than 2× of training length 200. Accuracy on languages outside of C-RASP (red) degrades rapidly beyond the training length.

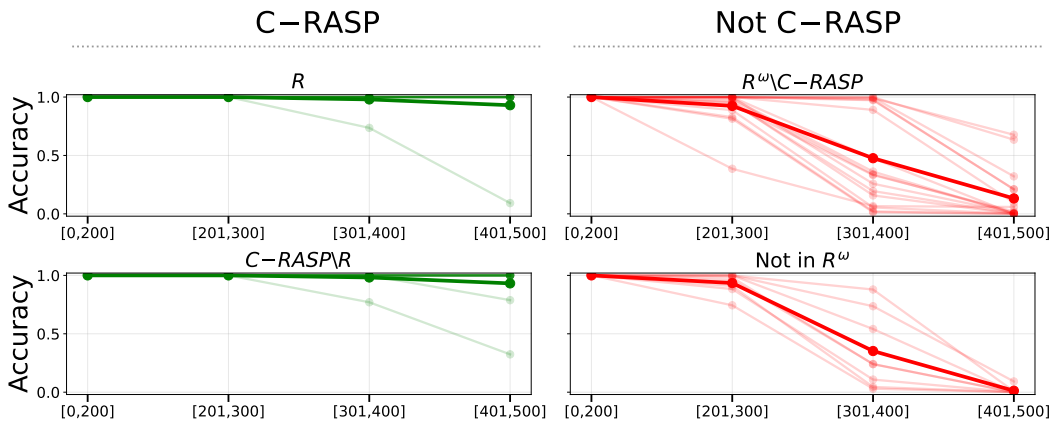


Figure 3: **Length generalization on regular languages (longer training length).** Models are trained on strings of lengths in $[l_{\min}, 200]$, and evaluated on bins $[l_{\min}, 200]$ (in-distribution) up to $[401, 500]$ in bins of width 100. Each curve corresponds to a language. Green curves denote languages in C-RASP, while red curves denote languages not in C-RASP. Languages in C-RASP maintain near-perfect accuracy well beyond the training range, whereas languages outside C-RASP exhibit rapid degradation, typically failing shortly after. Languages corresponding to this plot can be viewed in Table 2.

B.2 Length Generalization Across Seeds

We evaluate length generalization across multiple random model initializations. For each language, we compute the mean accuracy over five successful seeds, where a seed is considered successful if it achieves 100% accuracy on the in-distribution test set.

For languages not in C-RASP, performance is highly consistent across seeds, with all runs failing to generalize beyond the training range. In contrast, languages in C-RASP reliably generalize, though performance can vary across seeds - particularly when models are evaluated on lengths exceeding 2.5× the maximum training length.

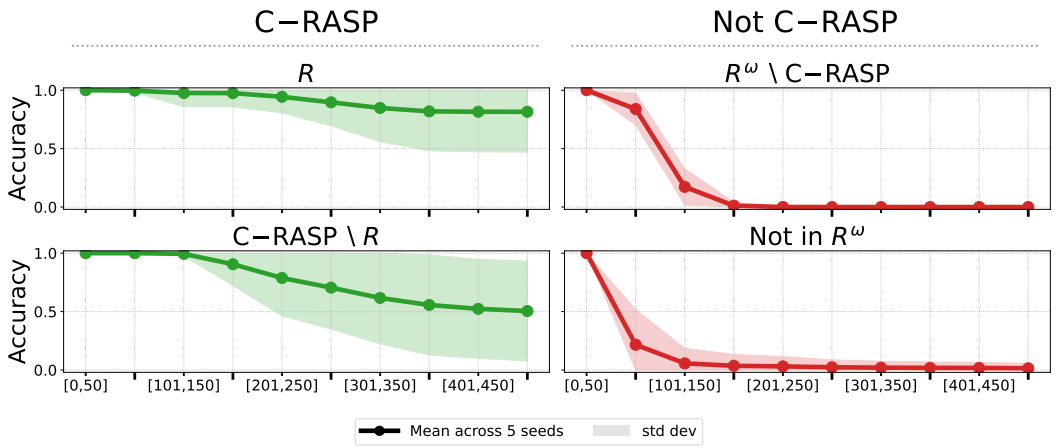


Figure 4: **Length generalization on regular languages (aggregated across seeds).** Models are trained on strings of lengths in $[\ell_{\min}, 50]$ and evaluated on bins from $[\ell_{\min}, 50]$ (in-distribution) up to $[451, 500]$ in steps of 50. Each curve shows the mean accuracy across 5 successful seeds, where a seed is considered successful if it achieves 100% accuracy on the in-distribution test data; shaded regions indicate standard deviation. Green curves correspond to languages in C-RASP, while red curves correspond to languages not in C-RASP. Results for individual best seeds are shown in Figure 2. Each panel includes all languages in the corresponding class from Table 1 in appendix B.4.

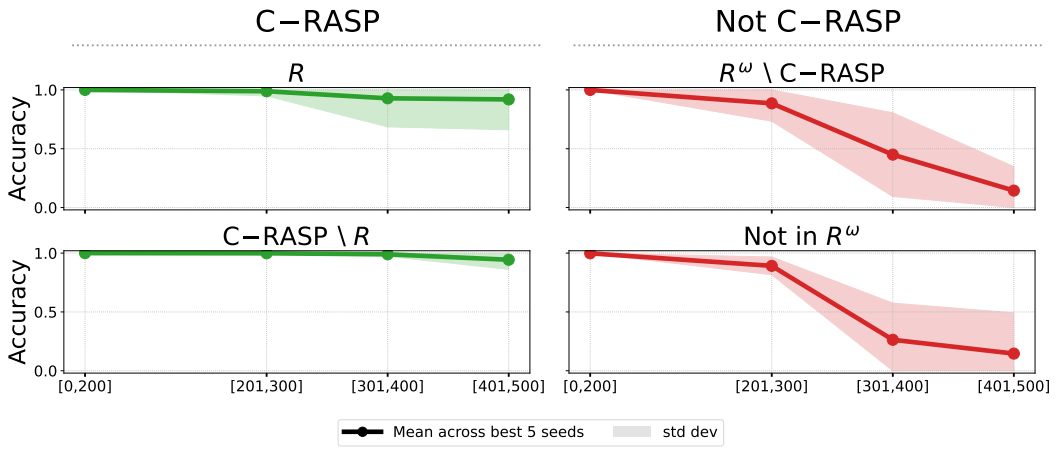


Figure 5: **Length generalization on regular languages (aggregated across seeds).** Models are trained on strings of lengths in $[\ell_{\min}, 200]$ and evaluated on bins from $[\ell_{\min}, 200]$ (in-distribution) up to $[401, 500]$ in steps of 100. Each curve shows the mean accuracy across 5 successful seeds, where a seed is considered successful if it achieves 100% accuracy on the in-distribution test data; shaded regions indicate standard deviation. Green curves correspond to languages in C-RASP, while red curves correspond to languages not in C-RASP. Results for individual best seeds are shown in Figure 3. Each panel includes all languages in the corresponding class from Table 2 in appendix B.4.

B.3 Increased Train Data Size

To investigate whether increased training data size can enable length generalization on languages not in C-RASP, we repeat our experiments with a substantially larger training set (100K instead of 10K).

Figure 6 shows the resulting length generalization behavior across language classes. We observe that increasing the training data does not qualitatively change the results. Languages

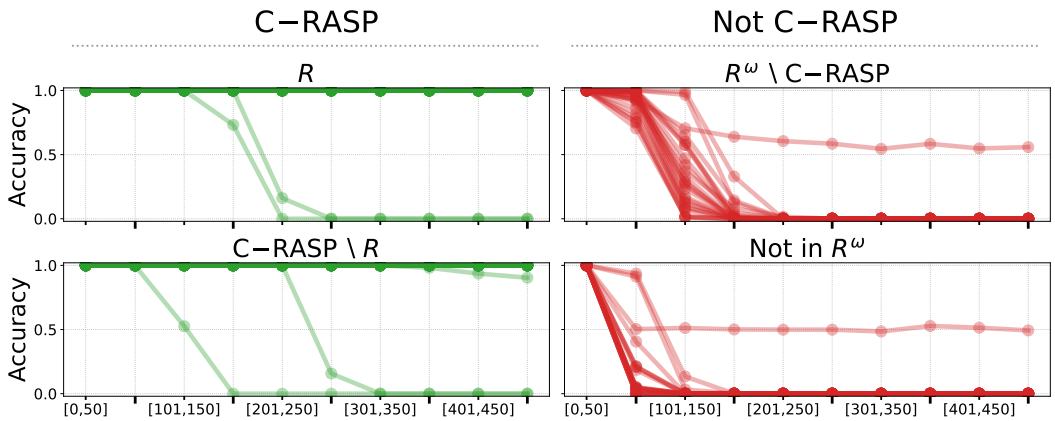


Figure 6: **Length generalization on regular languages with increased training data.** Models are trained on strings of lengths in $[\ell_{\min}, 50]$ using a larger training set (100K examples instead of 10K), and evaluated on bins from $[\ell_{\min}, 50]$ (in-distribution) up to $[401, 500]$ in steps of 50. Each curve responds to a language. Increasing the training data does not change trends in length generalization: languages in C-RASP continue to generalize, while languages not in C-RASP consistently fail to generalize beyond the training range. For these experiments we used a random subset of 50 languages from Table 1.

in C-RASP continue to mostly generalize beyond the training range, whereas languages not in C-RASP consistently fail to do so.

B.4 Languages

We report the complete set of regular languages used in our experimental evaluation in the main paper and appendix. Tables 1 and Table 2 provide a comprehensive overview of the dataset. For each language, we indicate membership in R, R^ω, and C-RASP.

C Algebraic Preliminaries

Definition 16 (Recognition; Syntactic Monoid)**.** *Let $L \subseteq \Sigma^*$ be a language. A monoid M* recognizes *L iff there is a homomorphism $h\colon \Sigma^* \to M$ and a subset $X \subseteq M$ such that $L = h^{-1}(X)$. The syntactic monoid $M(L)$ of a language L is the minimal monoid (up to isomorphism) which recognizes L.*

Definition 17 (Basic Units)**.** *The three basic semigroup units U_1, U_3, U_2 are given with their multiplication tables, where rows denote the first operand and columns denote the second.*

U_1	0	1
0	0	0
1	0	1

U_3	a	b
a	a	b
b	a	b

U_2	1	a	b
1	1	a	b
a	a	a	b
b	b	a	b

Definition 18 (Basic monoid operations)**.** *We define basic monoid operations.*

- *Submonoid. A monoid M is a submonoid of N (usually written $M \leq N$) whenever $M \subseteq N$ and M is closed under the monoid operation of N.*
- *Direct product. The direct product $M \times N$ of monoids M and N has elements (m, n) where $m \in M$ and $n \in N$, with the operation $(m_1, n_1) \cdot_{M\times N} (m_2, n_2) = (m_1 \cdot_M m_2, n_1 \cdot_N n_2)$.*
- *Homomorphism. A homomorphism of monoids $\phi\colon M \to N$ is a function such that $\phi(1_M) = 1_N$ and $\phi(m_1)\phi(m_2) = \phi(m_1 m_2)$.*
- *Division. A monoid M divides a monoid N (written $M \preceq N$) whenever there is a submonoid $X \leq N$ of N and a homomorphism $\phi\colon N \to M$ such that $M = \phi(X)$*

Formal Language	$\mathbf{R}$	$\mathbf{R}^\omega$	C-RASP	Formal Language	$\mathbf{R}$	$\mathbf{R}^\omega$	C-RASP
$(ca)^*(a)^*bb$	False	False	True	$(cc)^*cccb$	False	False	False
$bab(ba)^*$	False	False	True	$bccb(aa)^*$	False	False	False
$(ac)^*c+ba+a$	False	False	True	$((a)^*bb)^*$	False	False	False
$(cab+c)^*$	False	True	False	$(bb)^*ab+ac$	False	False	False
$ccc(ba)^*$	False	False	True	$bcbb(bb)^*$	False	False	False
$baca(bc)^*$	False	False	True	$(b)^*(b)^*(aa)*$	False	False	False
$(acba)^*$	False	False	True	$((a)^*)^*(cc)^*$	False	False	False
$a(bc)^*$	False	False	True	$((b)*ac)*$	False	False	False
$((b)^*)^*(ab)^*$	False	False	True	$acabcc(c)^*$	True	False	True
$(ca)^*(ab)^*$	False	False	True	$cbac(a)^*$	True	False	True
$(abcb)^*$	False	False	True	$bcac(a)^*+ba$	True	False	True
$(ac)^*ba(b)^*$	False	False	True	$caaba+(a)^*$	True	False	True
$(ca)^*(cb)^*$	False	False	True	$(a)^*caaa+c$	True	False	True
$(b)^*cc(ca)^*$	False	False	True	$b+ca+b(c+a)^*$	True	False	True
$(abbc)^*$	False	False	True	$(a)^*(a)^*b+aac$	True	False	True
$((a)^*)^*(ac)^*$	False	False	True	$bba(c+a)^*$	True	False	True
$(cb)^*caa+b$	False	False	True	$(b)^*b+aa+bba$	True	False	True
$(ba)^*cb+aa$	False	False	True	$(b)^*baa$	True	False	True
$(ba)^*(b)^*bc$	False	False	True	$cb(a)^*baa$	True	False	True
$bbbc(ca)^*$	False	False	True	$aca(a)^*$	True	False	True
$(bbac)^*$	False	False	True	$bc(b)^*abbb$	True	False	True
$c+cab+(bc)^*$	False	False	True	$aa(a)^*c$	True	False	True
$(baaa)^*$	False	False	True	$(a)^*a(b+b)^*$	True	False	True
$b+c+ac(ac)^*$	False	False	True	$a+c(a)^*cb+a$	True	False	True
$((ac)^*)^*$	False	False	True	$(b)^*cbbcaa$	True	False	True
$(a+c)^*cbbb$	False	True	False	$aa(c)^*+acca$	True	False	True
$(a+ac+a)^*$	False	True	False	$ccc(b)^*$	True	False	True
$((b)^*bc)^*$	False	True	False	$cb(a)^*+abbc$	True	False	True
$((b)^*ab)^*$	False	True	False	$bcba+(a)^*aa$	True	False	True
$((a)^*ac)^*$	False	True	False	$((b)^*b)^*$	True	False	True
$((a)^*ca)^*$	False	True	False	$(a)^*b+acab$	True	False	True
$(baa+b)^*$	False	True	False	$(b)^*cc(b)^*ca$	True	False	True
$((b)^*a+c)^*$	False	True	False	$(c)^*b+b(b)^*$	True	False	True
$(a+c)^*(cb)^*$	False	True	False	$(ab+aabb)^*$	False	True	False
$((a)^*ab)^*$	False	True	False	$(ab+bbaa)^*$	False	False	True
$(bab+b)^*$	False	True	False	$(a^+b^+)^+$	False	True	False
$(ba+a+c)^*$	False	True	False	$(a^+b^+)^k$	True	False	True
$(b+a)^*aaac$	False	True	False	$(ab)^+a^+$	False	False	True
$(cb(c)^*)^*$	False	True	False	$(ab)^+a^+b^+$	False	False	True
$(c+b(a)^*)^*$	False	True	False	$(ab)^+a^+b^+a^+$	False	False	True
$(b+ac+c)^*$	False	True	False	$(ab)^+b(ab)^+$	False	False	True
$(a+aac)^*$	False	True	False	$((ab)^+b^+)^k$	False	False	True
$(ab(a)^*)^*$	False	True	False	$((ab)^+b^+)^+$	False	True	False
$(ca(c)^*)^*$	False	True	False	$(ab+ba)^*$	False	False	True
$(ac(a)^*)^*$	False	True	False	$(ab+bba)^*$	False	False	False
$(cbb+b)^*$	False	True	False	$(ab+aabb)^*$	False	True	False
$(baa+a)^*$	False	True	False	$(\Sigma\setminus b0)^*a(\Sigma\setminus a,b1)^*$	False	True	False
$(cca+a)^*$	False	True	False	$(\Sigma\setminus a,b0)^*a(\Sigma\setminus b1)^*$	True	False	True
$(bc(c)^*)^*$	False	True	False	$(a(ab)^*b)^*$	False	False	True
$b+(bb)^*$	False	False	False	$(aa)^*$	False	False	False
$(a)^*(a)^*(bb)^*$	False	False	False	$(a\|b)^*bb(a\|b)^*$	False	True	False
$(aa+aa)^*$	False	False	False	$(cc)^*$	False	False	False
$(bb)^*cbac$	False	False	False	$\{abe\}^*be^*$	False	True	False
$(bb)^*(bb)^*$	False	False	False	$\{abe\}^*be^*b\{abe\}^*$	False	True	False
$(aa+(b)^*)^*$	False	False	False	$a^*(ba^*ba^*)*$	False	False	False
$aba+c(bb)^*$	False	False	False	$(ab)^*$	False	False	True
$(bb)^*(c)^*ac$	False	False	False	$\Sigma^*ab\Sigma^*$	True	False	True
$bbcc(aa)^*$	False	False	False	Σ^*b	False	True	False
$(ca)^*(cc)^*$	False	False	False	$\Sigma^*a\Sigma^*b\Sigma^*$	True	False	True
$cc+cc(bb)^*$	False	False	False	$\Sigma^*a\Sigma^*$	True	False	True
$(cc)^*ccb+b$	False	False	False	$b\Sigma^*$	True	False	True
$(ab)^*+(bb)^*$	False	False	False	$(\Sigma\setminus a1,b0)^*a1(\Sigma\setminus a2,b1)^*a2(\Sigma\setminus b2)^*$	True	False	True
$(cc)^*bac$	False	False	False	$(aa)^*bb(a)^*$	False	False	False
$aa+ca(aa)*$	False	False	False	$(b+bc)^*$	False	True	False

Table 1: Set of 130 regular languages used in the experiments in Figure 2 and Figure 4. For each language, we report membership in $\mathbf{R}$, $\mathbf{R}^\omega$, and C-RASP. For each language, we sampled 10K words for the training set (lengths 0–50) and 1K words for each evaluation length bin.

Definition 19 (Variety)**.** *A variety of monoids is a class closed under submonoids, division, and direct product. A variety of languages is a class closed under inverse homomorphism, Boolean operations, and shifts ($a^{-1}Lb^{-1}$).*

Formal Language	$\mathbf{R}$	$\mathbf{R}^\omega$	C-RASP
$(ab)^*bb(aabb)^*$	False	False	True
$(abb)^*(a)(c+a)(a)cbb+a^*(c)$	False	False	True
$(b^*ca+ca)(ac+b)(c+a)^*(b)^*+(bc)^*(b+c)(c)+a(c)^*caa$	False	False	True
$(b^*bc+bb)c(a)(a)c^*aa+(bc)^*(b)^*(b)b^*aa+(a+a)(b)aba+(c)^*c^*bc+b^*+bc+a$	False	False	True
$(cba)^*a$	False	False	True
$(cac)^*b(b+c)^*a^*+bb+a(b)a^*ba+(a)^*c^*bb$	False	False	True
$(cb)^*ab+(aa+a)^*a(a)bba$	False	False	True
$(ac+aa+c)(ca+b+b)(c+a)^*ba^*ab+(ab+c)^*(b+c)^*(b)^*a^*ac+c+b^*abc+b^*cb+bc+c+b$	False	False	True
$(c^*cc+b)(bc)^*(c+a)^*+(bc+a)^*+b$	False	False	True
$(acc)^*+(ab+c+b)(a+b)^*+(a+b)+cabc$	False	False	True
$(cac)^*(ca+c+c)(c)(b)^*ccc+(ac+c+a)^*a^*(c)^*ba+(a+a)^*(a)a$	False	False	True
$(bcb)^*(b)(b)^*(c)^*a^*$	False	False	True
$(a^*ac+ba+a+a)(ab+c)^*ba^*b^*bc$	False	False	True
$(bcb+c+c+c)(bc+c+b)(b+a)^*(a)^*bba$	False	True	False
$(a^*cc+bb+b)(ac)^*b^*+(c+b+b)^*(a)(b)a^*ba+(a+c)^*cc$	False	True	False
$(bcb+cb+a+c)^*$	False	True	False
$(ccb+ac+c+b)+(cc+b)^*(b+c)^*(c)cba+a^*$	False	True	False
$b^*+(cb+a)^*(c+b)^*c^*c^*c+(a+a)^*(c)b^*+(b)^*$	False	True	False
$(b^*c+ba)^*c^*+(aa+b+c)(a+c)a^*ccc$	False	True	False
$(a^*ab+cb+a+c)^*$	False	True	False
$(cbb+cb+a+a)(ca+b+c)^*(c+b)^*(b)^*b^*+(cb)^*(a+b)(b)+b(b)c^*ba$	False	True	False
$(b^*ab+ab+a+c)^*(cb+b)(b)^*+c^*(b+b)^*(c)^*$	False	True	False
$(a^*bb+ba+b+b)^*+b^*(b+a)(c)acb+c(b)b^*b+aaba+cc$	False	True	False
$(c^*bb+bb+c)+(ab+c+c)^*ab+(b+c)^*a+ab$	False	True	False
$(c^*+bb+c+a)(ac+a)c(c)a^*ab+(cb+b+a)(a)a^*c+(c+b)^*(b)b^*a$	False	True	False
$(b^*ca+cb+b)^*(a+a)(b+a)a$	False	True	False
$(baa+ab+b)^*$	False	True	False
$(c^*+ac+a+b)^*b^*(c+b)$	False	True	False
$(a^*cb+ca+b+a)c+a^*c^*c^*+(b+a)^*(a)^*b^*ac+(b)a^*c$	False	True	False
$a^*+(aa+a+c)(a)(b)^*b+(b+a)^*(a)bba$	False	True	False
$(abb+b)(ba+c+c)(c+a)+(b)(a+b)^*+(c)b^*c^*+(a)^*a^*a$	True	False	True
$(b^*cc+bb+b+a)(ca+b)$	True	False	True
$(b^*b+ca+b+c)(a+a+c)^*(a+c)+(aa+c)a(a)$	True	False	True
$(a^*)^*$	True	False	True
$(aca)^*(a+c+a)^*+(ba+b+a)^*cc^*c^*+(c)(b)^*b+b^*aab+b$	True	False	True
$(cbc)(bc)a^*(c)^*a^*ab+a^*(a+b)(c)c^*bb+c^*(c)^*a^*aa$	True	False	True
$b^*(ba+c)c^*c^*+c(b+b)^*bccc+(c)a^*c^*bb+a^*c^*cc$	True	False	True
$b+b^*(c)(b)cca+ac^*baa+(b)^*ac+b^*cb+b+c+b$	True	False	True
$(acc+c+c+b)(a)(a+a)^*(a)^*bbb+(ba+a+b)+a^*ca^*aa+(b)+acc$	True	False	True
$a^*b^*+a(b+a)(a)b^*ac$	True	False	True
$(a^*b)^*+(ac+b+a)b^*bc^*cb$	True	False	False
$(a^*ac)+c^*b(c)^*+(c+b)^*+(b)aac$	True	False	True
$a^*+(a)^*$	True	False	True
$(a+bb)^*a^*$	False	False	False
$(b^*bc)+(aa+c)^*$	False	False	False
$(cc+aa+a+b)(aa+b+b)^*+b^*b+(b+b)(c)aca+(a)^*acb+cab$	False	False	False
$(b+aa+b+c)^*$	False	False	False
$b(aa+c)^*(b)(b)cca$	False	False	False
$b^*+(aa+c)^*(a)^*(c)^*bbb$	False	False	False
$(b^*cc)^*$	False	False	False

Table 2: Set of 50 regular languages used in the experiments in Figure 3 and 5. For each language, we report membership in $\mathbf{R}$, $\mathbf{R}^\omega$, and C-RASP. For each language, we sampled 10K words for the training set (lengths 0–50) and 1K words for each evaluation length bin.

Definition 20. *Define* $\mathbf{V}\circ\mathbf{W}$ *as the variety generated by all* $V\circ W$ *where* $V \in \mathbf{V}$ *and* $W \in \mathbf{W}$*. Define* $\mathrm{wp}^1(\mathbf{V}) = \mathbf{V}$, $\mathrm{wp}^{k+1}(\mathbf{V}) = \mathrm{wp}^k(\mathbf{V})\circ\mathbf{V}$*, and* $\mathrm{wpc}(\mathbf{V}) = \bigcup_{k>0}\mathrm{wp}^k(\mathbf{V})$*. We also define* $\mathrm{wp}^k(M)$ *for monoids, taking* $\mathrm{wp}^1(M)$ *as the variety generated by* M*.*

We define some varieties that we use in the paper

Definition 21. *The following are standard, except for* $\mathbf{Dy}$*.*

- $\mathbf{R}$ *is the variety of* $\mathcal{R}$*-trivial monoids (Brzozowski & Fich, 1980)*
- $\mathbf{A}$ *is the variety of all aperiodic monoids*
- $\mathbf{G}$ *is the variety of all finite groups*
- $\mathbf{Dy}$ *is the variety generated by* $M(\mathcal{D}_k)$ *for all* $k \in \mathbb{N}$

D Causally-masked programs

In this section we abstract from C-RASP to the class of *causally-masked programs*, which intuitively are the class of straight-line programs in which each operation at position i can

only depend on previously defined operations and positions $j \leq i$. This content follows Thérien & Wilke (2001).

Definition 22 (Causally-masked programs)**.** *Causally-masked programs are those with the syntax*

$$\phi ::= \sigma \mid \neg\phi_1 \mid \phi_1 \wedge \phi_2 \mid \mathcal{O}\langle \phi_1, \phi_2, \ldots, \phi_k \rangle$$

where $\sigma \in \Sigma$ and $\mathcal{O}$ is some operator of arity k. For each operator there is an associated collection $K_{\mathcal{O}} \subseteq \Sigma^* \times (2^{\mathbb{N}})^k$ of words and sets of positions contained in the operator. Semantics are defined

$$\begin{aligned}
w, i \models \sigma &\iff w_i = \sigma \\
w, i \models \neg\phi &\iff w, i \not\models \phi \\
w, i \models \phi_1 \wedge \phi_2 &\iff w, i \models \phi_1 \text{ and } w, i \models \phi_2 \\
w, i \models \mathcal{O}\langle \phi_1, \phi_2, \ldots, \phi_k \rangle &\iff (w_{\leq i}, \{j \mid w, j \models \phi_1\}, \ldots, \{j \mid w, j \models \phi_k\}) \in K_{\mathcal{O}}
\end{aligned}$$

This is just defining Lindström quantifiers, which can be instantiated by the typical logics as follows.

Example 23. *Linear Temporal Logic is the class of Causally-masked programs with the binary* **since** *operator, typically written infix.*

$$K_{\textbf{since}} = \{(w, R_1, R_2) \mid \exists k \in R_2 \text{ st } k < |w| \text{ and } [k, |w|) \subseteq R_1\}$$

In this way

$$\begin{aligned}
w, i \models \phi_1 \textbf{ since } \phi_2 &\iff (w_{\leq i}, \{j \mid w, j \models \phi_1\}, \{j \mid w, j \models \phi_2\}) \in K_{\textbf{since}} \\
&\iff \text{there exists } k < i \text{ such that } w, k \models \phi_2 \text{ and } w, j \models \phi_1 \text{ for all } k < j < i
\end{aligned}$$

Similarly, C-RASP *is the class of causally-masked programs with the operator* $(\Lambda, C) = \sum_{1 \leq m \leq k} \lambda_m \cdot \overleftarrow{\#}\phi_m \geq C$

$$K_{(\Lambda, C)} = \left\{ (w, R_1, R_2, \ldots, R_k) \middle| \sum_{1 \leq m \leq k} \lambda_m \cdot |R_m| \geq C \right\}$$

In this way

$$\begin{aligned}
w, i \models \sum_{1 \leq m \leq k} \lambda_m \cdot \overleftarrow{\#}\phi_m \geq C &\iff (w_{\leq i}, \{j \mid w, j \models \phi_1\}, \ldots \{j \mid w, j \models \phi_2\}) \in K_{(\Lambda, C)} \\
&\iff \sum_{1 \leq m \leq k} \lambda_m \cdot |\{j \mid w, j \models \phi_m\}| \geq C
\end{aligned}$$

Definition 24. *Let Φ and Ψ be classes of causally-masked programs formulas over alphabet Γ and Σ, respectively. Let $G = \{\psi_\gamma\}_{\gamma \in \Gamma}$ be a family of formulas in Ψ. Let θ_G be a mapping of formulas given by*

$$\begin{aligned}
\theta_G(Q_\gamma) &\mapsto \psi_\gamma \\
\theta_G(\neg\phi) &\mapsto \neg\theta_G(\phi) \\
\theta_G(\phi_1 \wedge \phi_2) &\mapsto \theta_G(\phi_1) \wedge \theta_G(\phi_2) \\
\theta_G(\mathcal{O}\langle \phi_1, \phi_2, \ldots, \phi_k \rangle) &\mapsto \mathcal{O}\langle \theta_G(\phi_1), \theta_G(\phi_2), \ldots, \theta_G(\phi_k) \rangle)
\end{aligned}$$

We write $\phi[\gamma \mapsto \psi_\gamma]$ for this substitution. This is called a Ψ substitution of Φ formulas. We write $\Phi \star \Psi$ for the class of all Ψ substitutions of Φ formulas. The semantics are as would be expected. By default we let this operation be right-associative.

To help formalize the expressivity of causally-masked programs, we define a class of languages.

Definition 25 (End-Pointed Language)**.** *A pointed word is a tuple (w, p) for $w \in \Sigma^*$ and $1 \leq p \leq |w|$. An end-pointed language is a set of pointed words where the point denotes the end of the prefix of the string used for recognition.*

- *For Φ a class of causally-masked programs, $P(\Phi)$ is the set of pointed languages L for which there exists $\phi \in \Phi$ such that $(w, p) \in L$ iff $w, p \models \phi$.*
- *For $\mathbf{M}$ a class of typed monoids, $P(\mathbf{M})$ is the set of pointed languages L for which there exists $(M, \mathfrak{T}_M, \mathcal{E}_M) \in \mathbf{M}$, typed homomorphism $h\colon \Sigma^* \to (M, \mathfrak{T}_M, \mathcal{E}_M)$, a type $\mathfrak{M} \in \mathfrak{T}_M$, and a finite set $C \subseteq M$ such that $(w, p) \in L$ iff $(h(w_1 w_2 \cdots w_{p-1}), h(w_p)) \in \mathfrak{M} \times C$.*

Programs are classically connected to algebraic characterizations of languages via wreath product principles. We define how to take two classes of programs and obtain a more complex class.

Definition 26 (Program composition)**.** *Let Φ, Ψ be classes of causally-masked programs. The class $\Phi \star \Psi$ consists of all programs in Φ where atomic operations may refer to programs in Ψ.*

For instance, $\mathsf{C\text{-}RASP}_1 \star \mathsf{C\text{-}RASP}_1$ is equivalent to the class of depth 2 programs $\mathsf{C\text{-}RASP}_2$. We will see in appendix E.2 that causally-masked programs can be closely connected to wreath products of monoids.

E Typed Monoids

Krebs (2008) developed a framework for using infinite monoids to recognize languages. We present a restriction of the aforementioned framework to the case of wreath products (a one-sided version of the block product used in previous work), which ultimately provides an exact algebraic characterization of C-RASP. The core issue here is that the wreath product of infinite monoids can generate uncountably many elements, which can be too powerful.

Proposition 27. *Consider the classic wreath product $\mathbb{Z} \circ \mathbb{Z}$. Then $M(L) \preceq \mathbb{Z} \circ \mathbb{Z}$ for every L.*

Proof. Without loss of generality let $\Sigma = \{a, b\}$. Consider the submonoid of $\mathbb{Z} \circ \mathbb{Z}$ generated by the image of Σ^* under the homomorphism $\sigma \mapsto (f_\sigma, 1)$ where $f_a(x) = 2^x$ and $f_b(x) = 0$. In essence, this creates a mapping $w \mapsto (f_w, |w|)$ where $f_w(0)$ outputs a unique integer for each w. Thus, $\mathbb{Z} \circ \mathbb{Z}$ can recognize arbitrary languages. □

This problem motivates the definition of *typed monoids*, which restricts the accepting sets.

E.1 Definitions

Definition 28. *A typed monoid is a triple $(M, \mathfrak{T}_M, \mathcal{E}_M)$ where M is a finitely generated monoid, $\mathfrak{T}_M$ is a finite Boolean algebra over M, and $\mathcal{E}_M$ is a finite subset of M. Elements of $\mathfrak{T}_M$ are the* types *and elements of $\mathcal{E}_M$ are the* units*. A language L is recognized by $(S, \mathfrak{T}_M, \mathcal{E}_M)$ if there exists a homomorphism $h\colon \Sigma^* \to S$ such that $h(\Sigma) \subseteq \mathcal{E}_M$ and $L = h^{-1}(\mathfrak{M})$ for some $\mathfrak{M} \in \mathfrak{T}_M$.*

As an example, the language MAJORITY can be recognized by the typed monoid $(\mathbb{Z}, \{(-\infty, 0], [1, \infty)\}, \{-1, 1\})$ via the type $[1, \infty)$ and the homomorphism $a \mapsto 1$ and $b \mapsto -1$. We will typically refer to this typed monoid as $\mathbb{Z}$.

We define morphisms

Definition 29. *Let $(S, \mathfrak{T}_S, \mathcal{E}_S)$ and $(T, \mathfrak{T}_T, \mathcal{E}_T)$ be typed monoids. A typed monoid homomorphism $h\colon (S, \mathfrak{T}_S, \mathcal{E}_S) \to (T, \mathfrak{T}_T, \mathcal{E}_T)$ is a triple $(h_S, h_{\mathfrak{T}_S}, h_{\mathcal{E}_S})$ such that:*

- *$h_S\colon S \to T$ is a monoid homomorphism*
- *$h_{\mathfrak{T}_S}\colon \mathfrak{T}_S \to \mathfrak{T}_T$ is a homomorphism of Boolean algebras*
- *$\forall \mathfrak{S} \in \mathfrak{T}_S, h_{\mathfrak{T}_S}(\mathfrak{S}) = h_{\mathfrak{T}_S}(\mathfrak{S}) \cap h_S(S)$*
- *$\forall \mathfrak{s} \in \mathcal{E}_S, h_S(\mathfrak{s}) = h_{\mathcal{E}_S}(\mathfrak{s})$*

And due to the compatibility of $h_S, h_{\mathfrak{T}_S}, h_{\mathcal{E}_S}$ we can omit the subscripts. We say that a typed monoid $(S, \mathfrak{T}_S, \mathcal{E}_S)$ recognizes the language $L \subseteq \Sigma^*$ if there is a morphism $h\colon \Sigma^* \to S$ with $h(\Sigma) \subseteq \mathcal{E}_E$ and a type $\mathfrak{S} \in \mathfrak{T}_S$ such that $L = h^{-1}(\mathfrak{S})$.

So we want our functions to be compatible with the finite types, which motivates the following definition. In a sense, this requires that all elements of the same type, up to some constant shifting C, behave the same under the function.

Definition 30 (Type-respecting Functions)**.** *Let S be a set and $(T, \mathfrak{T}_T, \mathcal{E}_T)$ be a typed monoid and let $C \subseteq T$ be a nonempty finite set of constants. A function $f : T \to S$ is called type respecting with respect to $(T, \mathfrak{T}_T, \mathcal{E}_T)$ and C if the preimage $f^{-1}(s)$ can be described by a finite Boolean combination of conditions of the form $tc \in \mathfrak{T}$ where c is a constant in T (not necessarily in $\mathcal{E}_T$) and $\mathfrak{T} \in \mathfrak{T}_T$.*

Intuitively, the image of x under a type-respecting function depends only on the type of xc for some qualified set of constants c. Now the typed wreath product is similar to the untyped case, though the functions are constrained to be type-respecting functions.

Definition 31 (Typed Wreath Product)**.** *Let $(M, \mathfrak{T}_M, \mathcal{E}_M)$, $(N, \mathfrak{T}_N, \mathcal{E}_N)$ be two typed monoids, $C \subseteq N$ be a finite set. The typed wreath product*

$$(U, \mathfrak{T}_U, \mathcal{E}_U) = (M, \mathfrak{T}_M, \mathcal{E}_M) \circ_C (M, \mathfrak{T}_N, \mathcal{E}_N)$$

of $(M, \mathfrak{T}_M, \mathcal{E}_M)$ with $(N, \mathfrak{T}_N, \mathcal{E}_N)$ is defined such that

- *$\mathcal{E}_U$ consists of all elements (f, n), where $t \in \mathcal{E}_N$, and $f : N \to \mathcal{E}_M$ is a type respecting function (see definition 30) with respect to $(N, \mathfrak{T}_N, \mathcal{E}_N)$ and C*
- *U is the submonoid of $M \circ N$ generated by $\mathcal{E}_U$*
- *$\mathfrak{T}_U$ consists of all types $\mathfrak{U}_{\mathfrak{M},\mathfrak{N}} = \{(f, n) \in U \mid f(1_M) \in \mathfrak{M}, n \in \mathfrak{N}\}$, where $\mathfrak{M} \in \mathfrak{T}_M$, $\mathfrak{N} \in \mathfrak{T}_N$*

Definition 32 (Typed Monoid Variety)**.** *A typed monoid variety is a class of typed monoids closed under*

- *Division*
- *Shifting (changing types by inverse multiplication)*
- *Unit relaxation (swapping out units)*
- *Trivial extension (applying a congruence)*

Definition 33 (Typed Wreath Product Closure)**.** *For typed monoid varieties define $\mathbf{V} \circ \mathbf{W}$ as the variety generated by all $V \odot W$ where $V \in \mathbf{V}$ and $W \in \mathbf{W}$. Define $\mathrm{wp}^1(\mathbf{V}) = \mathbf{V}$, $\mathrm{wp}^{k+1}(\mathbf{V}) = \mathrm{wp}^k(\mathbf{V}) \odot \mathbf{V}$, and $\mathrm{wpc}(\mathbf{V}) = \bigcup_{k>0} \mathrm{wp}^k(\mathbf{V})$. We also define $\mathrm{wp}^k((M, \mathfrak{T}_M, \mathcal{E}_M))$ for typed monoids $(M, \mathfrak{T}_M, \mathcal{E}_M)$, taking $\mathrm{wp}^1((M, \mathfrak{T}_M, \mathcal{E}_M))$ as the variety generated by $(M, \mathfrak{T}_M, \mathcal{E}_M)$.*

Finally, we note the compatibility of the classical wreath product and the typed wreath product, which will become important in our algebraic decision procedure.

Lemma 34. *Assume $M \preceq S, N \preceq T$ where M, N are finite and S, T are typed. Then*

$$M \circ N \preceq S^N \odot T \tag{1}$$

where $\circ$ is the classical wreath product and $\odot$ is the typed wreath product.

Proof. Call these typed semigroups $(S, \mathfrak{T}_S, \mathcal{E}_S)$ and $(T, \mathfrak{T}_T, \mathcal{E}_T)$. Let $h_M \colon (S', {\mathfrak{T}_S}', {\mathcal{E}_S}') \to (M, 2^M, M)$ and $h_N \colon (T', \mathfrak{T}_{T'}, {\mathcal{E}_T}') \to (N, 2^N, N)$ define the divisions. We will define a function $h \colon (U, \mathfrak{T}_U, \mathcal{E}_U) \to M \circ N$ where $(U, \mathfrak{T}_U, \mathcal{E}_U) \leq ({S^N}, {\mathfrak{T}_S}^N, {\mathcal{E}_S}^N) \odot (T, \mathfrak{T}_T, \mathcal{E}_T)$. First, let U be the submonoid of $(S^N)^T \times T$ generated by (f_d, t) where f_d for $d \in (S')^N$ is defined such that $f_d(1_T) = d$ and $f_d(t) = [n \mapsto d(n + h_N(t))]$ for all $t \in T'$, and $f_d(t) = 1_{(S')^N}$ for $t \in T \setminus T'$. Note that all (f, t) in U satisfy the following constraints:

1. $t \in T'$
2. $\mathrm{Im}(\mathrm{Im}(f)) \subseteq S'$
3. $f(t_1 + t_2)(n) = f(t_1)(n + h_N(t_2))$
4. $f(t_1) = f(t_2)$ whenever $h_N(t_1) = h_N(t_2)$
5. $\mathrm{Im}(f(t)) = 1_{S'^N}$ for $t \in T \setminus T'$

As a submonoid of the wreath product, $\mathfrak{T}_U$ consists of all types $\mathfrak{U}_{\mathfrak{S}\mathfrak{N},\mathfrak{T}} = \{(f, t) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}, t \in \mathfrak{T}\}$, where $\mathfrak{S}\mathfrak{N} \in {\mathfrak{T}_{S'}}^N$, $\mathfrak{T} \in \mathfrak{T}_{T'}$ and 1_T is the neutral element of T. Let $\mathcal{E}_U$ consist of (f, t) where $t \in {\mathcal{E}_T}'$. Define a function $h \colon (U, \mathfrak{T}_U, \mathcal{E}_U) \to M \wr N$ such that $h((f, t)) = (g, h_N(t))$ where $g(n) = h_M(f(1_T)(n))$.

- All functions in U are type-respecting. This is because each of the generators f_d is type-respecting – since h_N is a homomorphism on types $h_N \colon \mathfrak{T}_T \mapsto 2^N$, the image $f_d(t)$ is determined by a boolean combination of conditions on the type of t.
- h is a surjection, because any $(g, n) \in M \wr N$:
  - There is $f \in U$ such that $g(\cdot) = h_M(f(t)(\cdot))$, because all $d \in (S')^N$ are represented in U. We just pick d to be compatible with g and f_d witnesses the preimage. As for which d to choose, we construct d where for $n_0 \in N$, we pick an element $s_0 \in h_M^{-1}(g(n_0))$ (which exists by the surjectivty of h_M) and set d such that $d(n') = s_0$ for all n' where $g(n') = g(n_0)$.
  - There is $t \in T'$ such that $h_N(t') = n$ by the surjectivity of h_N.
- h is a homomorphism on elements. From (3) we get that $f(t_1 + t_2)(n) = f(t_1)(n + h_N(t_2))$.

$$
\begin{aligned}
&h((f_1, t_1)(f_2, t_2)) \\
&= h((f_1 +^{t_1} f_2, t_1 t_2)) \\
&= (n \mapsto h_M((f_1 +^{t_1} f_2)(1_T)(n)), h_N(t_1 t_2)) \\
&= ((n \mapsto h_M(f_1(1_T)(n))) + (n \mapsto h_M(^{t_1} f_2(1_T)(n))), h_N(t_1 t_2)) \\
&= ((n \mapsto h_M(f_1(1_T)(n))) + (n \mapsto h_M(f_2(1_T + t_1)(n))), h_N(t_1 t_2)) \\
&= ((n \mapsto h_M(f_1(1_T)(n))) + (n \mapsto h_M(f_2(1_T)(n + h_N(t_1)))), h_N(t_1) h_N(t_2) \quad \text{by (3)} \\
&= ((n \mapsto h_M(f_1(1_T)(n))) +^{h_N(t_1)} (n \mapsto h_M(f_2(1_T)(n))), h_N(t_1) h_N(t_2) \\
&= (n \mapsto h_M(f_1(1_T)(n)), h_N(t_1))(n \mapsto h_M(f_2(1_T)(n)), h_N(t_2)) \\
&= h((f_1, t_1)) h((f_2, t_2))
\end{aligned}
$$

- h is a homomorphism on types. Let $\mathfrak{U}_{\mathfrak{S}\mathfrak{N},\mathfrak{T}}$ be a type of $(U, \mathfrak{T}_U, \mathcal{E}_U)$. Here, $h(\mathfrak{S}\mathfrak{N}) \in 2^{M^N}$ and $h(\mathfrak{T}) \in 2^N$, because h_M, h_N are homomorphisms on types by assumption[3]. First, h respects complements

$$
\begin{aligned}
h(\overline{\mathfrak{U}_{\mathfrak{S}\mathfrak{N},\mathfrak{T}}}) &= h(\{(f, t) \in U \mid f(1_T) \notin \mathfrak{S}\mathfrak{N} \text{ or } t \notin \mathfrak{T}\}) \\
&= \{h((f, t)) \in M \wr N \mid h(f(1_T)) \notin h(\mathfrak{S}\mathfrak{N}) \text{ or } h(t) \notin h(\mathfrak{T})\} \\
&= 2^{M^N} \times 2^N \setminus \{h((f, t)) \in M \wr N \mid h(f(1_T)) \in h(\mathfrak{S}\mathfrak{N}), h(t) \in h(\mathfrak{T})\} \\
&= \overline{h(\mathfrak{U}_{\mathfrak{S}\mathfrak{N},\mathfrak{T}})}
\end{aligned}
$$

And h respects union

$$
\begin{aligned}
&h(\mathfrak{U}_{\mathfrak{S}\mathfrak{N}_1,\mathfrak{T}_1} \cup \mathfrak{U}_{\mathfrak{S}\mathfrak{N}_1,\mathfrak{T}_1}) \\
&= h(\{(f, t) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_1, t \in \mathfrak{T}_1\} \cup \{(f, t) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_2, t \in \mathfrak{T}_2\}) \\
&= \{h((f, t)) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_1, t \in \mathfrak{T}_1\} \cup \{h((f, t)) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_2, t \in \mathfrak{T}_2\} \\
&= h(\{(f, t) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_1, t \in \mathfrak{T}_1\}) \cup h(\{(f, t) \in U \mid f(1_T) \in \mathfrak{S}\mathfrak{N}_2, t \in \mathfrak{T}_2\}) \\
&= h(\mathfrak{U}_{\mathfrak{S}\mathfrak{N}_1,\mathfrak{T}_1}) \cup h(\mathfrak{U}_{\mathfrak{S}\mathfrak{N}_1,\mathfrak{T}_1})
\end{aligned}
$$

And h preserves the inf and sup

$$
\begin{aligned}
h(\emptyset) &= \emptyset \\
h(U) &= \{h(f_d, t) \mid d \in (S')^N, t \in T'\} \\
&= \{(h_M(d), h_N(t)) \mid d \in (S')^N, t \in T'\} \\
&= M^N \times N
\end{aligned}
$$

□

[3]Note that Krebs (2008) defined the type of (f, t) independent of t (in order to ease the connection to logic), in which case the image under h would not be a type in $2^{M^N} \times 2^N$. By conditioning the type on t, the homomorphism on types goes through without need for additional direct products to enforce the types of T.

E.2 Proof of Typed Wreath Product Principle

First, we show how the type of an element is computed within the wreath product. Here we write π_1 and π_2 as the projections from the first and second coordinates of $M^N \times N$

Lemma 35. *Let $h\colon \Sigma^* \to (T, \mathfrak{T}_T, \mathcal{T}) = (M, \mathfrak{T}_M, \mathcal{E}_M) \circ_C (N, \mathfrak{T}_N, \mathcal{E}_N)$ for $C \subseteq N$. Let $\pi_1(\mathfrak{T}) \in \mathfrak{T}_M$ be such that $t \in \mathfrak{T} \iff \pi_1(t)(1_N) \in \pi_1(\mathfrak{T})$. Then $h(w) \in \mathfrak{T} \in \mathfrak{T}_T$ iff*

$$\sum_{1 \le i \le |w|} \pi_1(h(w_i)) \left(\prod_{1 \le j < i} \pi_2(h(w_j)) \right) \in \pi_1(\mathfrak{T}) \in \mathfrak{T}_M$$
$$\prod_{1 \le j < \le i} \pi_2(h(w_j)) \in \pi_2(\mathfrak{T}) \in \mathfrak{T}_N$$

Proof. We walk through the computation. First, we note that $h(w) \in \mathfrak{T}$ iff $\pi_1(h(w)) \in \pi_1(\mathfrak{T})$, and this type $\pi_1(\mathfrak{T})$ exists by the definition of the typed wreath product. Let $h(w_i) = (f_i, n_i)$ where $f_i \in M^N$ and $n_i \in N$. Then compute

$$\begin{aligned}
h(w) &= h(w_1)h(w_2)\dots h(w_{|w|}) \\
&= (f_1, n_1)(f_2, n_2)\dots(f_{|w|}, n_{|w|}) \\
&= \left(f_1 + {}^{n_1} f_2 + \dots + {}^{\left(\prod_{1 \le j \le |w|-1} n_j\right)} f_{|w|}, \prod_{1 \le i \le |w|} n_i \right) \\
&= \left(\sum_{1 \le i \le |w|} {}^{\left(\prod_{1 \le j < i} n_j\right)} f_i, \prod_{1 \le i \le |w|} n_i \right)
\end{aligned}$$

Recall by the definition of the wreath product that ${}^n f(x) = f(xn)$, so ${}^n f(1_N) = f(n)$.

$$\begin{aligned}
\pi_1(h(w))(1_N) \in \pi_1(\mathfrak{T}) &\iff \sum_{1 \le i \le |w|} {}^{\left(\prod_{1 \le j < i} n_j\right)} f_i(1_N) \in \pi_1(\mathfrak{T}) \\
&\iff \sum_{1 \le i \le |w|} f_i \left(\prod_{1 \le j < i} n_j \right) \in \pi_1(\mathfrak{T}) \\
&\iff \sum_{1 \le i \le |w|} \pi_1(h(w_i)) \left(\prod_{1 \le j < i} \pi_2(h(w_j)) \right) \in \pi_1(\mathfrak{T})
\end{aligned}$$

The computation of the second coordinate is routine as is computed in $(N, \mathfrak{T}_N, \mathcal{E}_N)$

$$\pi_2(h(w)) \in \pi_2(\mathfrak{T}) \iff \prod_{1 \le j < \le i} \pi_2(h(w_j)) \in \pi_2(\mathfrak{T}).$$

□

Now connection between the algebraic and logical formulations is often spelled out using statements in the form of a "wreath product principle".

Theorem 36 (Typed Wreath Product Principle)**.** *Let Φ, Ψ be classes of causally-masked programs and $\mathbf{M}, \mathbf{N}$ be varieties of monoids such that $L(\Phi) = L(\mathbf{M})$ and $P(\Psi) = P(\mathbf{N})$. Then $L(\Phi \star \Psi) = L(\mathbf{M} \circ \mathbf{N})$.*

Proof.

- Suppose $h\colon \Sigma^* \to (T, \mathfrak{T}_T, \mathcal{T}) = (M, \mathfrak{T}_M, \mathcal{E}_M) \circ_C (N, \mathfrak{T}_N, \mathcal{E}_N)$ for $C_N \subseteq N$. For each type $\mathfrak{T}_{\mathfrak{M},\mathfrak{N}} \in \mathfrak{T}_T$ we will construct a formula $\theta_{\mathfrak{T}} \in \Phi \star \Psi$ such that $w \models \theta_{\mathfrak{T}} \iff h(w) \in \mathfrak{T}$. Observe from Lemma 35 that $h(w) \in \mathfrak{T}$ iff

$$\sum_{1 \le i \le |w|} \pi_1(h(w_i)) \left(\prod_{1 \le j < i} \pi_2(h(w_j)) \right) \in \mathfrak{M} \qquad \text{and} \qquad \prod_{1 \le j < \le i} \pi_2(h(w_j)) \in \mathfrak{N}.$$

  The first coordinate can be viewed as recognition of a language by $\mathfrak{M}$ in M via a homomorphism $h_1\colon ({\mathfrak{T}_N}^C)^* \to (M, \mathfrak{T}_M, \mathcal{E}_M)$. By assumption obtain a $\phi \in \Phi$ that recognizes this language. As for the word $w' \in ({\mathfrak{T}_N}^C)^*$ we have that

$$w'_i = (\mathfrak{N}_{c_1}, \mathfrak{N}_{c_2}, \ldots, \mathfrak{N}_{c_{|C|}}) \iff \bigwedge_{c \in C} \left[\left(c \prod_{1 \le j < i} \pi_2(h(w_j)) \right) \in \mathfrak{N}_c \right]$$

  Observe that for each c, we use $(N, \mathfrak{T}_N, \mathcal{E}_N)$ to recognize an end-pointed language accepting with type $\mathfrak{N}_c$. Then by assumption we obtain $\psi_c \in \Psi$ recognizing each prefix, and then use the substitution

$$\phi_{\mathfrak{M}} \left[(\mathfrak{N}_{c_1}, \mathfrak{N}_{c_2}, \ldots, \mathfrak{N}_{c_{|C|}}) \mapsto \bigwedge_{c \in C} \psi_c \right]$$

  The second coordinate can be viewed as recognition of a language by $\mathfrak{N}$ in N. Since $\mathbf{N}$ is a variety, $L(\mathbf{N}) \subseteq P(\mathbf{N})$. We obtain a formula $\psi_{\mathfrak{N}} \in \Psi$ such that recognizes the same language and define $\theta_{\mathfrak{T}} = \phi_{\mathfrak{M}} \wedge \psi_{\mathfrak{N}}$. This results in a formula $\theta_{\mathfrak{T}} \in \Phi \star \Psi$ which recognizes the same language as $(T, \mathfrak{T}_T, \mathcal{T})$.
- The other direction is similar. Suppose we have a formula with substitution $\phi[\gamma \mapsto \psi_\gamma] \in \Phi \star \Psi$. Because $P(\Psi) = P(\mathbf{N})$, there are typed monoids N_γ that can compute the substitution's output γ at each position. Then because $L(\Phi) = L(\mathbf{M}$, we obtain a typed monoid M that can recognize the resulting language over Γ. The entire computation can thus be computed using a monoid in $\mathbf{M} \circ \mathbf{N}$.

□

F C-RASP

Here we reiterate some of the definitions of C-RASP, which can be found in prior papers by Yang & Chiang (2024); Huang et al. (2025).

F.1 Definitions

Definition 37. *The syntax of* C-RASP *formulas is defined:*

$$\phi ::= \sigma \mid \neg\phi_1 \mid \phi_1 \wedge \phi_2 \mid \sum_{t \in \mathcal{T}} \alpha_t \cdot \overleftarrow{\#}[\phi_t] \sim k$$

where $\sigma \in \Sigma$, $\alpha_i, k \in \mathbb{Z}$ and $\sim \in \{<, \le, =, \ge, >\}$. The semantics of formulas is defined as follows:

$$\begin{aligned}
w, i \models \sigma &\iff w_i = \sigma \\
w, i \models \neg\phi &\iff w, i \not\models \phi \\
w, i \models \phi_1 \wedge \phi_2 &\iff w, i \models \phi_1 \text{ and } w, i \models \phi_2 \\
w, i \models \sum_{t \in \mathcal{T}} \alpha_t t \sim k &\iff \sum_{t \in \mathcal{T}} \alpha_t \cdot |\{j \in [1, i] \mid w, j \models \phi\}| \sim k.
\end{aligned}$$

We write $w \models \phi$ iff w<EOS>$, |w| + 1 \models \phi$ where <EOS> $\notin \Sigma$ is a special end-of-sequence symbol. and we say that ϕ defines the language $L(\phi) = \{w \mid w \models \phi\}$.

In the sequel, we will use a *DAG* (directed acyclic graph) representation of C-RASP formulas, where a subformula φ may be used *multiple times* in a formula. Such a formula can be thought of as a straight-line *program*, i.e., a sequence $\varphi = (\varphi_i)_{i=1}^n$, where φ_i is any C-RASP definition that could refer to φ_j with $j < i$.

Definition 38. *The syntax of* C-RASP *is as follows:*

$$\phi ::= \sigma \mid t_1 < t_2 \mid \neg\phi_1 \mid \phi_1 \wedge \phi_2 \qquad \sigma \in \Sigma \qquad \textit{Boolean-valued formulas}$$
$$t ::= \overleftarrow{\#}[\phi_1] \mid t_1 + t_2 \mid 1 \qquad \textit{integer-valued terms}$$

The semantics of formulas is defined as follows:

$$w, i \models \sigma \iff w_i = \sigma \tag{3a}$$
$$w, i \models \neg\phi \iff w, i \not\models \phi \tag{3b}$$
$$w, i \models \phi_1 \wedge \phi_2 \iff w, i \models \phi_1 \textit{ and } w, i \models \phi_2 \tag{3c}$$
$$w, i \models t_1 < t_2 \iff t_1^{w,i} < t_2^{w,i}. \tag{3d}$$

The semantics of terms is defined as follows:

$$\overleftarrow{\#}[\phi]^{w,i} = |\{j \in [1, i] \mid w, j \models \phi\}| \tag{4a}$$
$$\overrightarrow{\#}[\phi]^{w,i} = |\{j \in [i, |w|] \mid w, j \models \phi\}| \tag{4b}$$
$$(t_1 + t_2)^{w,i} = t_1^{w,i} + t_2^{w,i} \tag{4c}$$
$$1^{w,i} = 1. \tag{4d}$$

We write $w \models \phi$ if $w, |w| \models \phi$, and we say that ϕ defines the language $L(\phi) = \{w \mid w \models \phi\}$.

The table below shows how this program works for the string $(())()$, which belongs to $\mathcal{D}_2$.

predicate	definition	description	a	a	b	b	a	b
a		is left paren	$\top$	$\top$	$\bot$	$\bot$	$\top$	$\bot$
b		is right paren	$\bot$	$\bot$	$\top$	$\top$	$\bot$	$\top$
C_a	$:= \overleftarrow{\#}[a]$	num of left parens	1	2	2	2	3	3
C_b	$:= \overleftarrow{\#}[b]$	num of left parens	0	0	1	2	2	3
ϕ_{low}	$:= C_a - C_b \geq 0$	depth above 0	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
ϕ_{up}	$:= C_a - C_b \leq 2$	depth below 2	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
ϕ_{bounded}	$:= \phi_{\text{low}} \wedge \phi_{\text{up}}$	depth bounded	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
ϕ_{matched}	$:= \#[\neg\phi_{\text{bounded}}] = 0$	depth bounded everywhere	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
ϕ_{balanced}	$:= C_a = C_b$	balanced at end	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\top$
$\phi_{\mathcal{D}_2}$	$:= \phi_{\text{matched}} \wedge \phi_{\text{balanced}}$	balanced at end	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\top$

The table below shows how this program works for the string $())()($, which does not belong to $\mathcal{D}_2$.

predicate	definition	description	a	b	b	a	b	a
a		is left paren	$\top$	$\bot$	$\bot$	$\top$	$\top$	$\bot$
b		is right paren	$\bot$	$\top$	$\top$	$\bot$	$\top$	$\bot$
C_a	$:= \overleftarrow{\#}[a]$	num of left parens	1	1	1	2	2	3
C_b	$:= \overleftarrow{\#}[b]$	num of left parens	0	1	2	2	3	3
ϕ_{low}	$:= C_a - C_b \geq 0$	depth above 0	$\top$	$\top$	$\bot$	$\top$	$\bot$	$\top$
ϕ_{up}	$:= C_a - C_b \leq 2$	depth below 2	$\top$	$\top$	$\top$	$\top$	$\top$	$\top$
ϕ_{bounded}	$:= \phi_{\text{low}} \wedge \phi_{\text{up}}$	depth bounded	$\top$	$\top$	$\bot$	$\top$	$\bot$	$\top$
ϕ_{matched}	$:= \#[\neg\phi_{\text{bounded}}] = 0$	depth bounded everywhere	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
ϕ_{balanced}	$:= C_a = C_b$	balanced at end	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\top$
$\phi_{\mathcal{D}_2}$	$:= \phi_{\text{matched}} \wedge \phi_{\text{balanced}}$	balanced at end	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$

F.2 C-RASP as a variety of Languages

Proposition 39. C-RASP *defines a variety of languages.*

Proof. We sketch the proof here.

- Closed under Boolean combinations: by definition.

- Closed under inverse homomorphisms: follow the construction in Yang et al. (2025, Lemma F.2) or Huang et al. (2025, Lemma 34), but ignoring positional predicates and allowing arbitrary symbols in the image of the homomorphism.
- Closed under factors: Let L be in C-RASP, and $a \in \Sigma$. To obtain $a^{-1}L$, we can detect the beginning in C-RASP and simulate the computations at a prefix a. To obtain La^{-1}, at <EOS> we simulate the computations at a<EOS>.

□

Because the langauges form a variety, there must exist a corresponding class of monoids (Behle et al., 2011, Theorem 2).

Remark 40. *If we defined recognition without the* <EOS> *symbol, the result would not be a variety, because of the special role played by the final position. For instance, a* C-RASP *program can separate* $\{a,b\}^*b$ *from* $\{a,b\}^*a$*, but taking the inverse of a homomorphism that deletes* e *and keeps* a, b *unchanges leads to the two sets* $\{a,b\}^*be^*$ *from* $\{a,b,e\}^*ae^*$ *which no* C-RASP *program can separate.*

F.3 Existing Characterizations

Known regular language membership results

Example 41. *We know the following language characterizations*

1. $\mathbf{R} \subset$ C-RASP $\cap \mathbf{REG} \subset \mathbf{A}$ *(Huang et al., 2025, Lemma 38).*
2. Σ^*b*, is not in* C-RASP *(Huang et al., 2025, Lemma 38).*
3. $\Sigma^*bb\Sigma^*$*is not in* C-RASP*, because it is not in the larger class* $\widehat{\mathsf{MAJ}}_2[<]$ *by Lemma 6.11 in Krebs (2008)*
4. *For any* k*,* $(a^+b^+)^k$ *in* C-RASP *by Yang et al. (2025)*
5. $(ab)^+$ *in* C-RASP

Proposition 42. *The following from Krebs (2008):*

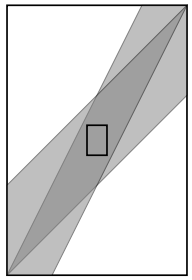
- C-RASP $\subseteq \widehat{\mathsf{MAJ}}_2[<] \subseteq \mathbf{DA}\Box\mathbf{G}$
- C-RASP $\subseteq \widehat{\mathsf{MAJ}}_2[<] \subseteq \overline{\mathbf{Ab}}$

Lemma 43. *Paraphrase from Krebs (2008). If we can find a family of Parikh rectangles which accommodates an unbounded family of inner rectangles reachable using prefixes/suffixes of a desired form, set* λ *to be that family of prefixes and* ϱ *that family of suffixes, and then we can recognize* $\lambda^{-1}L\varrho^{-1}$ *in lower depth.*

For a formula defining c *half-planes, we can get an inner rectangle of size* $n_a \times n_b$ *inside a rectangle of size* $3^{n_a} \times 3^{n_b}$*, so it suffices that the families are unbounded.*

Proposition 44. $(ab + bba)^* \notin \widehat{\mathsf{MAJ}}_2[<]$

Proof. For $(ab + bba)^*$, use say a family of 2×3 rectangles. Prefixes and suffixes of the form $(ab + bba)^*$ can reach anywhere within the gray slices in the picture. Restrict to strings where $3\#a = 2\#b$. Reduce to depth 1. Here the contradiction will be to put 3 a's in a row.



□

While we have some individual results, there is no exact characterization. The next section develops our exact characterization.

F.4 Algebraic Characterization of C-RASP

Theorem 11. $L(\text{C-RASP}) = L(\text{wpc}(\mathbb{Z}))$

Proof. First, we note that $\text{wpc}(\mathbb{Z})$ and $\text{wp}^2(\mathbb{Z})$ are varieties of typed monoids by definition. Then, we establishing that $P(\text{C-RASP}_1) = P(\text{wp}^2(\mathbb{Z}))$, noting that C-RASP is an instance of a causally-masked program. First, by (Yang et al., 2025, Lemma A.4) every C-RASP program can be written such that it only counts over positions $j < i$, with Boolean operations testing the symbol at i. The strict counting can be translated into a typed monoid over $\text{wp}^2(\mathbb{Z})$ (since the equations in C-RASP may not just be $\overleftarrow{\#}\phi \geq 0$, we require another wreath product to add constants into the equation), and the testing of the symbol at i can be handled by constants in $\text{wp}^2(\mathbb{Z})$. The other direction is similar. Thus by appendix E.2, $L(\text{C-RASP}_1) = L(\mathbb{Z} \circ \mathbb{Z})$. From this base case we can build up to $L(\text{C-RASP}_k) = L(\text{wp}^k(\mathbb{Z}))$ by induction. □

G Derived Categories

Here we provide a formal exposition of the ideas that were informally presented in the body of the paper. As mentioned above, we will principally be interested in finite categories for use as algebraic objects.

Definition 45 (Category)**.** *A category X consists of a set of objects $Obj(X)$ and for each $c, c' \in Obj(X)$ a homset of arrows $X(c, c')$, often written as $x\colon c \to c'$ for $x \in X(c, c')$. A category is endowed with the following algebraic structure:*

- *For arrows we an associative composition operation, where $s\colon c \to c'$ and $t\colon c' \to c''$, compose into an arrow $st\colon c \to c''$. Furthermore, for $s\colon c_1 \to c_2$, $t\colon c_2 \to c_3$, and $v\colon c_3 \to c_4$, we have that $(st)v = s(tv)$.*
- *For objects c we have an identity arrow $1_c\colon c \to c$, where $s1_c = s$ and $1_c t = t$ for $s\colon c' \to c$ and $t\colon c \to c''$.*

We will think of a monoid as a single-object category with the monoid elements as arrows of the category. Between categories we can define relations, which do not necessarily have to be functions.

Definition 46 (Category relation)**.** *Let X, Y be categories. A category relation $f\colon X \to Y$ has*

- *An object relation $f\colon Obj(X) \to Obj(Y)$, thought of as a subset of $X \times Y$.*
- *For any corresponding edge sets $X(c, c')$ and $Y(d, d')$ where $d \in cf, d' \in c'f$, an edge set relation $f\colon X(c, c') \to Y(d, d')$.*

Each category relation f determines a subcategory $\#f$ of $X \times Y$ where

- $Obj(\#f) = \{(c, d)\colon d \in cf\}$
- $\#f[(c, d), (c', d')] = \{(x, y) \mid x \in X(c, c'), y \in Y(d, d'), y \in xf\}$

The most important kind of relation for us will be a relational morphism $\phi\colon X \triangleleft Y$. In a sense, this is a generalization of a homomorphism where you can take any function on the atomic elements of the X as a generator of the resulting relation $X \triangleleft Y$ after we close under composition in X.

Definition 47 (Relational Morphism)**.** *A* relational morphism *is a category relation where the object relation is a function and each hom-set relation is fully-defined. A relational morphism where the hom-set relations are injective is called a* division*.*

The notion of homomorphism and division for finite monoids is a special case of the definitions above for categories. Then, as discussed above, the analogue of the kernel of a group homomorphism (and thus the analogue of a "divisor") for monoids is the *derived category*. The worked example in section 4.2 hopefully provides some intuition on the structure of the derived category.

Definition 48 (Derived Category)**.** *Let $\phi\colon M \triangleleft N$ be a relational morphism between monoids. The* derived category *D_ϕ of the relational morphism is defined with $Obj(D_\phi) = \phi(S)$ and $Hom(n_1, n_2) = \{n_1 \to_{(m,n)} : \phi^{-1}(n_1) \to \phi^{-1}(n_2) \mid (m,n) \in \#\phi, n_1 n = n_2\}$. Composition of arrows is given by $n_0 \to_{(m_1,n_1)} n_0 n_1 \to_{(m_2,n_2)} = n_0 \to_{(m_1 m_2, n_1, n_2)}$.*

For readers familiar with the corresponding construction for groups, we provide some intuition. In some sense, $\ker\phi$ for $\phi\colon G \to H$ records what information is lost when compressing G into H. By recording what elements collapse into 1_H, we can reconstruct how every other component of G collapses into H (taking advantage of the inverses in the group to form connections between elements). Then, taking the quotient $G/(\ker\phi)$ precisely records what information is lost by this collapse, and by enriching H with this information again we reconstruct G via the division $G \preceq (\ker\phi) \circ H$.

In the case of monoids, we lack the nice closure properties of groups, and thus $\ker\phi$ cannot be used to reconstruct the collapsing behavior of every other component of G. So the corresponding structure must be enriched with additional information. Indeed, in the derived category construction, for each element of $n \in N$ we consider subsets of M which collapse into n via a relational morphism, and then must record specific information about the interactions of elements within and between these subsets. The derived category just stores the essential amount of information in order to reconstruct M via a wreath product $M \preceq V \circ N$, formalized by the Derived Category Theorem.

Theorem 49 (Derived Category Theorem)**.**

1. *Let $\phi\colon M \triangleleft N$ be a relational morphism of monoids, and let V be a monoid satisfying $D_\phi \preceq V$. Then there is a division of monoids $\theta\colon M \preceq V \circ N$.*
2. *Let $\theta\colon M \preceq V \circ N$ be a division of monoids, and let $\phi = \theta\pi\colon M \triangleleft N$ be the associated relational morphism. Then $D_\phi \preceq V^N$.*

H Algebraic Decision Procedure

H.1 Relevant Lemmas

Lemma 50. *If $M \preceq S$ and $N \preceq T$ then $M \wr N \preceq S \wr T$.*

Proof. This is a standard fact which we sketch out here. For some $S' \leq S$ and $T' \leq T$ there exists surjections $h_M\colon S' \to M$ and $h_N\colon T' \to N$. Let G' be the subset of $S^T \times T$ generated by all (f, t') where $t' \in T'$, $\mathrm{Im}(f) \subseteq S'$, and $f(t) = 1_S$ for $t \notin T'$, with the wreath product action inherited from $S \wr T$. Define $h\colon G \to M \wr N$ by $h(f, t') = (f', h_N(t'))$ where $f'(n) = h_M(f'(t))$ for some $t \in h_N^{-1}(n)$ (the choice does not matter). It is clear that h is a surjective homomorphism via inheritance from h_M and h_N. □

H.2 Decidability Proof (via Finite Derived Categories)

The following formalizes a basic construction in the theory of derived categories; it permits going from a derived category to a wreath product decomposition:

Definition 51 (Extension)**.** *Let M, S, T be finite monoids. Given relational morphisms $\phi : M \triangleleft T$, $\phi' : D_\phi \triangleleft S$, we define*

$$\mathrm{Ext}(\phi', \phi)(m) = \{(f, t) \in S^T \times T : t \in \phi(m); \forall x \in Obj(D_\phi) : f(x) \in \phi'(x \to_{(m,t)} xt)\}$$

Remark 52. *This construction is made at the top of p. 116, case (a) of the proof of Theorem 5.2 of Tilson (1987). There, it is also proven that it is a relational morphism. It is also proven that if ϕ' is a division, then so is $\mathrm{Ext}(\phi', \phi)$.*

Remark 53. *We write $Ext(\phi_1, \phi_2, \ldots, \phi_n)$ for $\mathrm{Ext}(\phi_n, \mathrm{Ext}(\phi_{n-1}, \ldots))$.*

The following is shown in Tilson (1987):

Proposition 54. $\mathrm{Ext}(\phi', \phi)$ *is a well-defined relational morphism* $M \triangleleft S \wr T$.

Theorem 55. *Given a finite monoid M, the procedure below correctly determines if M divides an iterated typed wreath product of $(\mathbb{Z}, \mathbb{Z}_+)$.*

We now state our decision procedure:

Definition 56 (Decision Procedure)**.** *Our input is a finite monoid M. Let $R_1, R_2, \ldots, R_k$ be the $\mathcal{R}$-classes of M, such that $R_i \prec R_j \Rightarrow i < j$. For $i = 1, 2, \ldots, k$, we maintain relational morphisms ϕ_i from M to an iterated wreath product of $\mathcal{D}_k$ and U_1. We maintain the invariant that*

$$\{m\} = \phi_i^{-1}(\phi_i(m)), \forall m \in R_1, \ldots, R_i \qquad (\mathrm{Invariant}(i))$$

Let N_0 be the trivial monoid and let $\phi_0 \colon M \triangleleft N_0$ be the trivial relational morphism. For $i = 1, \ldots, k$, do

1. *Note: In the automaton-based version, we first relabel transitions based on entry points. This is an algebraic analogue; it records, for any path through the category going into R_i, which arrow was used for passing into it. We will later refer to this annotation as the "entry point coordinate". It intuitively tells us via which element of R_i we first entered it. Later, $E_{i,j}$ will have multiple SCCs (or bonded components, the term used by Tilson) corresponding to R_i, indexed by the different possible entry points (Lemma 60).*

   Let $\delta : Obj(D_{\phi_{i-1}}) \to \{0, 1\}$ given as

   $$\delta(o) = \begin{cases} 1 & \exists m \in \phi_{i-1}^{-1}(o) : m \succeq_R R_i \\ 0 & else \end{cases}$$

   Let $\partial_i = \{(o \to_{s,t} ot) : \delta(o) = 0, \delta(ot) = 1\}$.

   Define $\phi' : D_{\phi_{i-1}} \triangleleft B$, where B is the left-zero semigroup with identity $*$ adjoined, with left-zeros indexed by the elements of ∂_i[4], as the relational morphism generated by (i.e., the intersection of all relational morphisms satisfying):

   $$\phi'(o \to_{(s,t)} ot) \supseteq \begin{cases} \partial_i & \delta(o) = \delta(ot) = 1 \\ \{*\} & \delta(o) = \delta(ot) = 0 \\ \{(o \to_{(s,t)} ot)\} & \delta(o) = 0, \delta(ot) = 1 \end{cases} \tag{5}$$

   where we note $\delta(o) = 1, \delta(ot) = 0$ cannot occur.

   By definition, $* \notin \phi'(1 \to_{(s,o)} o)$ iff $s \succeq_R R_i$.

   We note that B is R-trivial; hence it divides an iterated wreath product of U_1.

   This is a relational morphism. Set $\phi_{i,1} := \mathrm{Ext}(\phi', \phi_{i-1})$.

2. *Build a sequence of relational morphisms $\phi_{i,2}, \phi_{i,3}, \ldots$ as follows:*

   (a) *For $Ext(\phi_{i,j}, \ldots, \phi_{i,1}, \phi_{i-1}) \colon M \triangleleft N_{i,j}$, we define a monoid congruence on $N_{i,j}$ generated by*

   $$(f_{i,j}, \ldots, f_{i,1}, f) \equiv (f_{i,j}, \ldots, f_{i,1}, g) \tag{6}$$

   whenever $f_{i,1}(1_{N_{i-1}}) \neq *$. We then define $E_{i,j} = D_{\phi_{i,j}/\equiv}$ where $\phi_{i,j} : M \triangleleft N_{i,j}/\equiv$. Note: Intuitively, in this category, all information from earlier annotation is removed once we have entered R_i. This is helpful for making our algorithm computable, because we can just ignore any complications arising from interactions between old and new annotation. It also means that our algorithm might generate decompositions that are non-minimal in depth.

   (b) *Consider the set $\Omega_{i,j}$ of relational morphisms $\phi' : E_{i,j} \triangleleft \mathcal{D}_k$ (for any k) such that for each $s \in R_i$ and any $t \in \phi_{i,j}(s)$, $\perp \notin \phi'(1 \to_{s,t} t)$. Note: In the automaton-based proof, the key requirement is for the relabelings to be balanced. Here, we formulate this in terms of avoiding $\perp$, without explicity talking about loops. If one instead defines $\phi' : E_{i,j} \triangleleft (\mathbb{Z}, \mathbb{Z}_+)$, one would demand that ϕ' has finite image on any such arrow.*

[4] $B := \partial_i \cup \{*\}$, $*b = b$, $rb = r$ whenever $r \in \partial_i$, $b \in B$.

(c) *If there is such a* $\phi' : E_{i,j} \lhd \mathcal{D}_k$ *that assigns unequal images to two arrows starting in* R_i *(see Definition 57 below) from the same hom-set, then we take such a* ϕ'*, extend it back to* $\widehat{\phi}' : D_{\phi_{i,j}} \lhd \mathcal{D}_k$ *and build* $\phi_{i,j+1} := \mathrm{Ext}(\widehat{\phi}', \phi_{i,j}) \colon M \lhd N_{i,j+1}$.

(d) *Otherwise, we terminate this inner loop. One possibility is that* $\phi_{i,j}^{-1}(\phi_{i,j}(m)) = \{m\}$ *for each* $m \in R_i$*. By induction, we have satisfied* Invariant(i)*. Then, set* $\phi_i := \phi_{i,j} \colon M \lhd N_i$ *and pass to the next iteration in the outer loop. Else, we exit from the entire algorithm and declare* failure.

We declare success *and return* N_k *if we iterated through all* $i = 1, \ldots, k$ *without ever declaring failure.*

Note: Motivation of this algorithm: The generic strategy would be to keep generating relational morphisms to $\mathbb{Z}_+$, but that would not suffice for decidability, because (i) we wouldn't know how often to iterate, and (ii) wouldn't know which types to use. We "guide" the process by (i) proceeding along R classes, (ii) considering morphisms to well-selected finite monoids, (iii) passing to $E_{i,j}$ instead of the more complicated $D_{\phi_{i,j}}$, and (iv) focusing on the sets $\Omega_{i,j}$.

Throughout, we write $1 \in Obj(E_{i,j})$ for the object arising from the identity element of $D_{\phi_{i,j}/\equiv}$.

Definition 57. *For any of the categories* $E_{i,j}$ *constructed in the decision procedure, we define the following notiions. Let* $\pi : E_{i,j} \prec M$ *be the canonical division.*

1. *Let* S_i *be the set of arrows* $o \to_{s,t} ot$ *such that there are arrows* $1 \to_{s',o} o$ *and* $ot \to_{s'',t''} ott''$ *such that* $ss's'' \in R_i$.
2. *We say an arrow* α *"ends" in* R_i *if there is an arrow* β *such that* $\beta\alpha$ *is defined and* $\beta\alpha \in \pi^{-1}(R_i)$.
3. *We say an arrow* α *"starts" in* R_i *if there is an arrow* β *such that* $\beta\alpha$ *is defined and* $\beta \in \pi^{-1}(R_i)$.
4. *We say an object* o *is "inside* R_i*" if* $Hom(1 \to o) \cap \pi^{-1}(R_i) \neq \emptyset$.

Lemma 58 (Correctness)**.** *If the algorithm succeeds, then* M *divides an iterated wreath product of* $\mathcal{D}_k$ *and* U_1*. It also divides an iterated wreath product of* $(\mathbb{Z}, \{\mathbb{Z}_+\})$*, of the form* $(((\ldots\mathbb{Z})\circ\mathbb{Z})\circ\mathbb{Z})$.

Proof. We refer to a wreath product of the form $(\ldots\circ\cdot)\circ\cdot)\circ\cdot)\circ\cdot$ as *left-associative*.

First, we obtain a relational morphism $\phi_k : M \lhd N$ where N divides a wreath product of U_1 and $\mathcal{D}_k$. By the invariant maintained in the Decision Procedure, ϕ_k is injective, making it a division.

We now need to explain why M also divides a product of the form $(((\ldots\mathbb{Z})\circ\mathbb{Z})\circ\mathbb{Z})$. First, by we can obtain a representation in terms of $\mathcal{D}_k \prec U_1\circ\mathbb{Z}$ and U_1, in the left-associative bracketing due to the associativity of the finite wreath product at the level of varieties (Tilson, 1987). We thus obtain a weak bracketing of $(U_1\circ\mathbb{Z})$ and U_1 by Lemma 34. We now consider, as a term inside this representation, a product of the form

$$M\circ(U_1\circ\mathbb{Z})$$

Recognition of a word of length N by this monoid is determined by the type of $\prod_{i=1}^{N}(f_i, (g_i, h_i))$, which is determined by

$$\left(\prod_{i=1}^{N} f_i \left(\prod_{j=1}^{i-1}(g_j, h_j)\right), \prod_{i=1}^{N}(g_i, h_i)\right)$$

where $f_i : U_1 \circ \mathbb{Z} \to M$; specifically, $f_i(g_0, h_0) \in M$ is determined by $g_0(e) \in U_1$ and the type of $h_0\kappa \in \mathbb{Z}$ (for some units or constants κ). Now the left component is

$$\prod_{i=1}^{N} f_i \left(\prod_{j=1}^{i-1} (g_j, h_j) \right)$$
$$= \underbrace{\prod_{i=1}^{N} f_i \left(\underbrace{\prod_{j=1}^{i-1} g_j \left(\prod_{k=1}^{j-1} h_k \right)}_{\in U_1}, \underbrace{\prod_{j=1}^{i-1} h_j}_{\in \mathbb{Z}} \right)}_{\in M}$$

But we can compute this same information via a morphism to $(M \circ U_1) \circ \mathbb{Z}$. Applying this iteratively, we obtain a left-associatively bracketed wreath product of $U_1 \prec \mathbb{Z}$ and $\mathbb{Z}$, thus, of $\mathbb{Z}$.

□

The following properties of $\omega \in \Omega_{i,j}$ are key:

Lemma 59 (Balance on Loops)**.** *Let $\omega \in \Omega_{i,j}$.*

1. *Let $u \in R_i$, $\rho \in M$ such that $u\rho = u$. Consider arrows*
$$1 \to_{(u,o)} o \to_{(\rho,t)} ot$$
in $E_{i,j}$. Then in fact $ot = o$, and $\omega(o \to_{(\rho,t)} o) = \{0_{\mathcal{D}_k}\}$ where $0_{\mathcal{D}_k}$ is the neutral element of $\mathcal{D}_k$.

2. *Let u be such that $R_i u = R_i$.*

   For $v \in R_i$, consider arrows
$$1 \to_{(v,o)} o \to_{(u,t)} ot$$
in $E_{i,j}$. Then $\omega(o \to_{(u,t)} ot)$ is a singleton.

Proof. For the first point, we have two claims to prove: that (a) $ot = o$, and that (b) the image is $\{0_{\mathcal{D}_k}\}$. We first note that (a) holds at $j = 1$: $u \in R_i$, $u\rho = u$ and $1 \to_{(u,o)} o \to_{(\rho,t)} ot$ in $E_{i,j}$, then in fact $ot = o$ by construction of $E_{i,1}$. We also note that, if (b) has been shown for $1, \ldots, j$, then (a) follows for $1, \ldots, j+1$. We thus need to perform the inductive step for (b). We consider the arrow $o \to_{(\rho,t)} o$. Assume $z \in \omega(o \to_{(\rho,t)} o)$ and $z \neq 0$. Now, for all $k \geq 0$, $u\rho^k = u$; hence, $\perp \in \omega(1 \to_{(u,o)} o)$. This is a contradiction to $\omega \in \Omega_{i,j}$.

For the second point, from $R_i u = R_i$, obtain ρ such that $vu\rho = v$. We consider arrows:

$$1 \to_{(v,o)} o \to_{(u,t)} ot \to_{(\rho,t')} ott'$$

By (1), in fact, $ott' = o$ and $\omega(o \to_{(u,t)} ot \to_{(\rho,t')} ott') = \{0_{\mathcal{D}_k}\}$. Because ω is a relational morphism and $\mathbb{Z}$ is a group, this entails $\omega(o \to_{(u,t)} ot)$ must be a singleton. □

We deduce the following structural properties of $E_{i,j}$, which are used both for termination and for completeness of the procedure. As foreshadowed when we defined the "entry-point coordinate" $\phi_{i,1}$, we find that R_i is reflected in $E_{i,j}$ in multiple strongly connected components (or "bonded components"), each indexed by a different entry arrow of the "boundary set" ∂_i we defined there:

Lemma 60 (Structure of $E_{i,j}$)**.** 1. *Let $\alpha := (o \to_{(s,t)} ot) \in \partial_i$, and let $m \in R_i$. Then there is exactly one $u \in Obj(E_{i,j})$ such that both of the following hold: (i) there is an arrow $1 \to_{(m,u)} u$, and (ii) the $\phi_{i,1}$ component of u is α.*

2. *Let* $\alpha = (1 \to_{(m,u)} u), \alpha' = (1 \to_{(m',u')} u')$ *be arrows in* $E_{i,j}$*, where* $m, m' \in R_i$*; and assume* u, u' *have the same* $\phi_{i,1}$ *component. Then there is* $\beta \in Hom(u, u')$ *such that* $\alpha' = \alpha\beta$*.*

Proof. We show this by induction over j. The claims are immediate at $j = 1$. For the first claim, we show the inductive step using Lemma 59. For any arrow

$$\alpha = (1 \to_{(m,o)} o)$$

with $m \in R_i$, Lemma 59.2 entails that ω is single-valued on all arrows $\beta = (o \to_{(m',t)} ot)$ with $mm' \in R_i$; hence, inductively, after passing to $E_{i,j+1}$, only a single arrow is derived from β.

The second claim follows by choosing n such that $m' = mn$; the first claim enforces the presence of an arrow covering n in $Hom(u, u')$. □

Lemma 61 (Termination and Computability)**.** *The inner loop always terminates. Further, in each step of the inner loop, we can computably bound the k needed.*

Proof. For any arrow $\alpha := (1 \to_{(m,\dots)} \dots)$ in $E_{i,1}$, where $m \in R_i$, we consider the $\mathbb{Z}$-module $\mathfrak{M}_\alpha$ of the restrictions of relational morphisms $\omega : E_{i,j} \triangleleft \mathcal{D}_k$ (any k) to the set of arrows starting and ending on R_i that are reachable from α, such that ω avoids $\perp$ on all such arrows. There is a canonical projection $\pi_\alpha : \Omega_{i,j} \to \mathfrak{M}_\alpha$.

We first need to show that (i) the number of such α's, and (ii) the dimension of each $\mathfrak{M}_\alpha$, is uniformly bounded across all j; this immediately follows from Lemma 60.1.

We, second, note that, when a new $\phi' : E_{i,j} \triangleleft \mathcal{D}_k$ is chosen, then its projection onto $\mathfrak{M}_\alpha$ needs to, at least for some α, be oustide of the span of the projections of the morphisms selected so far. Thus, to compute the next ϕ', we can computably find an element of $\mathfrak{M}_\alpha$ not in the span of the projections of the previously selected relational morphisms. As $\mathbb{Z}$ is Noetherian and $\sum_\alpha dim(|\mathfrak{M}_\alpha|)$ is bounded across j, this entails termination.

□

Lemma 62 (Completeness)**.** *If the procedure terminates with failure, M does not divide an iterated wreath product of* $(\mathbb{Z}, \mathbb{Z}_+)$*.*

Proof. Assume that at, after constructing $E_{i,j}$, the procedure terminates with failure.

For each object o inside R_i, we consider the strongly connected component (or "bonded component" in Tilson (1987)) of o:

$$Obj(C_o) := \{o' \in Obj(E_{i,j}) : Hom(o \to o') \neq \emptyset, Hom(o' \to o) \neq \emptyset\} \tag{7}$$

As above, let π be the canonical division $\pi\colon E_{i,j} \preceq M$. Within any homset $Hom(o' \to o'')$ in C_o, we merge any two arrows β, β' such that, in $E_{i,j}$, $Hom(1 \to o')\beta$ and $Hom(1 \to o')\beta'$ both are not in $\pi^{-1}(\bigcup_{j \leq i} R_j)$. By construction, C_o is a well-defined category, and $C_o \prec E_{i,j} \prec M$.

We note that each object o' in C_o must be in R_i, for otherwise, if there is o' not in R_i but with $Hom(o' \to o) \neq \emptyset$ we would have been able to set $\phi'(u \to_{(s,t)} u't) := \begin{cases} 1 & u't = o' \\ 0 & \text{else} \end{cases}$ in $\Omega_{i,j}$ and have ϕ' assign different images to $o \to_{(1_M,0)} o$ vs the (unique) element of $Hom(o \to o') \cdot Hom(o' \to o)$ ending outsie of R_i, i.e., the inner loop would not have terminated.

Failure of algorithm implies some C_o must be non-trivial Failure of the algorithm entails that some C_o contains a non-singleton homset, i.e., C_o is not trivial. This is seen as follows. Failure of the algorithm implies that there is $m \in R_i$ such that $\phi_{i,j}^{-1}(\phi_{i,j}(m)) \supsetneq \{m\}$. Now consider $m' \in M$, $m \neq m'$ such that there is some $\hat{o} \in \phi_{i,j}(m) \cap \phi_{i,j}(m')$. Let o be the corresponding element of $N_{i,j}/\equiv$. This means that, in $E_{i,j}$, there are distinct arrows $1 \to_{(m,\tilde{o})} o$ and $1 \to_{(m',o)} o$. Now we consider the $\phi_{i,1}$ coordinate of o ("entry point coordinate"); this is of the form $(u \to_{(s,t)} ut)$ corresponding to some arrow of $D_{\phi_{i-1}}$. By inductive hypothesis, $Hom(1 \to u)$ in that category had exactly one object $1 \to_{(s',u)} u$. Now let v be the corresponding entry object, which must be contained in C_o (by Lemma 60.1); then (by Lemma 60.2) there is an arrow $\alpha = 1 \to_{(s's,v)} v$ such that there are $\beta, \beta' \in Hom(v, o)$ with $1 \to_{(m,\tilde{o})} o = \alpha\beta$ and $1 \to_{(m',o)} o = \alpha\beta'$. We thus have shown $Hom(v, o)$ to be nonsingleton.

Setting up a division Choose as C'' one such C_o (it doesn't matter which one). Assume M divides an iterated wreath product of $(\mathbb{Z}, \mathbb{Z}_+)$, then so does C''. Consider a division of a minimum-depth wreath product of $\mathbb{Z}$, $\psi : C'' \prec \underbrace{\mathbb{Z} \odot \ldots \odot \mathbb{Z}}_{T \text{ times}}$ (here all $\mathbb{Z}$ are typed like $(\mathbb{Z}, \mathbb{Z}_+)$); in particular, because C'' is not trivial, $T \geq 1$. Let $\omega = \pi^{(T)} \circ \psi$; by definition, $\omega : C'' \triangleleft \mathbb{Z}$. Our goal is to create a division $\mu : C'' \prec \underbrace{\mathbb{Z} \odot \ldots \odot \mathbb{Z}}_{T-1 \text{ times}}$. This would be a contradiction to the minimality of T. As a consequence, C'' cannot divide *any* wreath product of $(\mathbb{Z}, \mathbb{Z}_+)$.

Note: The basic idea is to show that ω cannot have provided any useful information, and in particular gives the same values across different arrows in a given hom-set. Either it stays bounded on all arrows going into R_i (in which case ω provides as much information as an element of $\Omega_{i,j}$ – and the algorithm having terminated indicates that no such element would have been helpful), or it is unbounded (in which case we can pass to a limiting element $\pm\infty$, showing that ω contributes no information useful to $(\mathbb{Z}, \mathbb{Z}_+)$-based recognizers).

Case 1: ω stays bounded on all arrows ending in R_i inside C'' First, assume ω stays bounded on all arrows ending in R_i inside C'', with upper bound k. We now obtain $\hat{\omega} : C'' \triangleleft \mathcal{D}_k$ by (i) replacing all values $|\cdot| > k$ with $\perp$ (so it maps to $\mathcal{D}_k$)[5], and (ii) closing the result under composition to make it again a relational morphism. Importantly, all arrows ending inside R_i avoid $\perp$ under $\hat{\omega}$. We now extend $\hat{\omega}$ to $\tilde{\omega} : E_{i,j} \triangleleft \mathcal{D}_k$ in such a way as to keep $\perp \notin \tilde{\omega}(1 \to_{(s,t)} s)$ for $s \in R_i$.[6] We observe that $\tilde{\omega} \in \Omega_{i,j}$. The algorithm having terminated means that *$\tilde{\omega}$ must be constant on each homset of C''*.[7] Simultaneously, by Lemma 59b, *ω can attain only a single value (with value $< k$) within any non-singleton homset starting in R_i*.[8]. In fact, ω can only attain a single value on any homset.[9] We obtain the desired division μ as follows.

[5]Note that this truncation does not respect composition, but (ii) fixes this.

[6]To achieve this, we do the following. For each arrow starting or ending outside of C_o, we first assign it the image $\{0\}$. We then close under composition to obtain a relational morphism. Because C_o is a strongly connected component, no new values are added on any of its arrows. Now assume $\perp \in \tilde{\omega}(1 \to_{(s,t)} s)$ for $s \in R_i$, and consider any path multiplying out to this arrow. If it involves C_o at all (otherwise the image will just be zero), it factorizes into a path ending outside of C_o (its image is just $\{0\}$), a unique entry arrow (its image is initialized as $\{0\}$), and a path inside C_o (its image must avoid $\perp$). Overall, $\tilde{\omega}$ must output a value distinct from $\perp$.

[7]Assume that it gives distinct images on two arrows α, α' in the same homset. At least one of them needs to start and end in R_i, for otherwise they would have been merged in the construction of C''. Hence, both arrows need to start in R_i. But then the algorithm would not have terminated at this point yet.

[8]Assume ω attains two distinct values on an arrow α appearing in a non-singleton homset. Then it must attain two distinct values also on an arrow α' that starts and ends in C''. But this would contradict Lemma 59b.

[9]If ω attained more than one value on any homset, this would need to be a singleton homset, and it would consequently assign more than one value to ane arrow. But then the endpoint object cannot be in R_i (and hence not in C''), for otherwise o would be reachable from it and ω would have non-singleton image on $Hom(o \to o)$, which would be a contradiction.

Note that any object in the image of ψ has the form $(g, \dots)$ where $g : \mathbb{Z} \to \underbrace{(\mathbb{Z} \circ \dots \circ \mathbb{Z})}_{T-1 \text{ times}}$. For arrows where ut is in R_i, we define:

$$\mu(u \to_{(s,t)} ut) := \{g(\nu) : (g, \dots) \in \psi(u \to_{(s,t)} ut), \nu \in \omega(Hom(o \to u))\} \tag{8}$$

(note that ν here is in fact unique) which is a subset of $\underbrace{(\mathbb{Z} \circ \dots \circ \mathbb{Z})}_{T-1 \text{ times}}$. This is a relational morphism due to the uniqueness of ν.[10] Also, μ is injective because ψ is.[11] Finally, μ respects the type structure because ψ does.

Case 2: ω is unbounded on some arrow ending in R_i inside C'' Let $\alpha = \left(u_0 \to_{(s_0,t_0)} u_0 t_0\right)$ in C'' be an arrow ending in R_i which is associated with an infinite number of different values under ω. Without loss of generality, we may assume that $\sup \omega(\alpha) = +\infty$.

Note that any object in the image of ψ has the form $(g, \dots)$ where $g : \mathbb{Z} \to \underbrace{(\mathbb{Z} \circ \dots \circ \mathbb{Z})}_{T-1 \text{ times}}$. Because g arises from a composition of type-respecting functions, $g(\infty) := \lim_{x \to \infty} g(x)$ is well-defined and in $\mathbb{Z}$. We define

$$\mu(u \to_{(s,t)} ut) := \{g(\infty) : (g, \dots) \in \psi(u \to_{(s,t)} ut)\} \tag{9}$$

which is a subset of $\underbrace{(\mathbb{Z} \circ \dots \circ \mathbb{Z})}_{T-1 \text{ times}}$.

This is a relational morphism $\mu : C'' \triangleleft \underbrace{(\mathbb{Z} \circ \dots \circ \mathbb{Z})}_{T-1 \text{ times}}$: For arrows β, β' such that $\beta\beta'$ is defined, we have:

$$\begin{aligned}
\mu(\beta)\mu(\beta') =& \{g(\infty)h(\infty) : g..., h...\} \\
=& \{\lim_{x \to \infty} (g(x)h(x)) : g..., h...\} \\
=& \{\lim_{x \to \infty} (g(x)h(x\tau_1)) : g..., h...\} \\
\subseteq& \{G(\infty) : (G, \dots) \in \psi(\beta\beta')\} \\
=& \mu(\beta\beta')
\end{aligned}$$

It is also type-respecting because ψ is.

We need to show that it is injective. Consider two distinct arrows in the same homset in C'':

$$\beta_1 := u \to_{(s,t)} ut \qquad \beta_2 := u \to_{(s',t')} ut$$

[10] We consider the product of the images of two compatible arrows:

$$\begin{aligned}
&\mu(u \to_{(s,t)} ut)\mu(ut \to_{(s',t')} utt') \\
=&\{g(\nu)g'(\nu') : (g, \dots) \in \psi(u \to_{(s,t)} ut), \nu \in \omega(Hom(1 \to u)), (g', \dots) \in \psi(ut \to_{(s',t')} utt')\}
\end{aligned}$$

for unique $\nu \in \omega(Hom(1 \to u))$ and $\nu' \in \omega(Hom(1 \to ut))$. On the other hand, again by definition and by uniqueness of the ν's,

$$\mu(u \to_{(ss',tt')} utt') = \{g''(\nu) : (g'', \dots) \in \psi(u \to_{(ss',tt')} utt')\}$$

Now for the unique $\nu'' \in \omega(Hom(u \to ut))$, we have $\nu' = \nu\nu''$ because ω is a relational morphism. Because ψ is also a relational morphism, $g(\nu)g'(\nu') = g(\nu)g'(\nu\nu'') = g''(\nu)$ for some $(g'', \dots) \in \psi(u \to_{(ss',tt')} utt')$. Note that the above reasoning shows that we here have a general fact about the eliminability of a right-hand factor in a wreath product when it assigns constant and unique values on each homset in the category. In more abstract terms, we have shown that ω provides just as much information as a relational morphism to the trivial monoid (i.e., no information at all).

[11] Consider two arrows $u \to_{s,t} ut$, $u' \to_{s',t'} ut$ in C'' with overlapping images, $g_s(\nu) = g_{s'}(\nu')$. Now for any $\alpha \in Hom(o \to u)$, the product of α with these two arrows would thus have overlapping images even under ψ; hence, the arrows must have been the same.

If $\beta_1 \neq \beta_2$, that means there is $\lambda \in R_i \cap \pi^{-1}(Hom(1 \to u))$ such that $\lambda s \neq \lambda s'$.

We now construct $\rho_1, \rho_2 \in M$ such that

$$\alpha := 1 \to_{(\rho_1, u_0)} u_0 \to_{(s_0, t_0)} u_0 t_0 \to_{(\rho_2, t')} u = 1 \to_{(\rho_1 s_0 \rho_2, u)} u \tag{10}$$

in $Hom(1 \to u)$ satisfies $\lambda \in \pi^{-1}(\alpha)$.[12] Hence, $\alpha\beta \neq \alpha\beta'$ in $E_{i,j}$. In C'', $\alpha' := u_0 \to_{(s_0,t_0)} u_0 t_0 \to_{(\rho_2,t')} u$ satisfies $\alpha'\beta \neq \alpha'\beta'$. Now, we note that $\sup \omega(\alpha') = +\infty$; this enforces that $\mu(\beta) \cap \mu(\beta') = \emptyset$.

Overall, we have obtained a division μ in either case, and, by contradiction, established that M does not divide an iterated wreath product of $(\mathbb{Z}, \mathbb{Z}_+)$. □

Theorem 15. *Membership of $M \in$ C-RASP $\cap$ **REG** is decidable in $O(\mathsf{poly}(|M|))$ time.*

Proof. We need to show that (i) the algorithm from Definition 56 correctly determines membership in C-RASP (shown in Theorem 55), and (ii) that it runs in polynomial time in $|M|$. For (ii), we refer to the automaton-based formulation of the algorithm in Appendix J, with proof of time polynomial in the number of automaton states in Corollary 65.

□

I Algebraic Characterization (Necessary but not Sufficient Criterion)

Theorem 13. *$M \in \mathbf{R}^\omega$ iff M is aperiodic and every $\mathcal{R}$-class of M contains at most one idempotent.*

Proof. We note that $\mathbf{R}\circ\mathbf{G} \cap \mathbf{A}$ is exactly the aperiodic monoids with at most one idempotent in each $\mathcal{R}$-class. First, we show $\mathbf{R}^\omega \subseteq \mathbf{R}\circ\mathbf{G} \cap \mathbf{A}$. Suppose $M \in \mathbf{R}^\omega$. By substituting 1 for y in $\mathbf{R}^\omega$ we obtain that M satisfies

$$x^\omega x = x^\omega$$

which implies $M \in \mathbf{A}$ (Pin, 2009). Next, assume for sake of contradiction that $M \notin \mathbf{R}\circ\mathbf{G}$. Then there is a $\mathcal{R}$-class of M which contains at least 2 idempotents (Stiffler Jr, 1973, Theorem 3.18). Let $e_1 \neq e_2$ be $\mathcal{R}$-equivalent idempotents, where $e_1 = e_2 m_2$ and $e_2 = e_1 m_1$. Then

$$(e_1 e_2)^\omega = (e_1 e_1 m_1)^\omega = (e_1 m_1)^\omega = e_2^\omega = e_2$$

while

$$(e_1 e_2)^\omega e_1 = e_2 e_1 = e_2 e_2 m_2 = e_2 m_2 = e_1.$$

This implies $(e_1 e_2)^\omega \neq (e_1 e_2)^\omega e_1$, and thus $M \notin \mathbf{R}^\omega$, a contradiction. Thus $M \in \mathbf{R}\circ\mathbf{G}$.

Now we show $\mathbf{R}\circ\mathbf{G} \cap \mathbf{A} \subseteq \mathbf{R}^\omega$. Let $m_1, m_2 \in M$. Consider the idempotents $(m_1 m_2^\omega)^\omega$ and $(m_1 m_2^\omega)^\omega (m_2^\omega m_1)^\omega$. We show these are $\mathcal{R}$-equivalent.

First $(m_1 m_2^\omega)^\omega \geq_{\mathcal{R}} (m_1 m_2^\omega)^\omega m_1 \geq_{\mathcal{R}} (m_1 m_2^\omega)^{\omega+1}$, so by aperiodicity $(m_1 m_2^\omega)^\omega \equiv_{\mathcal{R}} (m_1 m_2^\omega)^\omega m_1$. Then

$$\begin{aligned}(m_1 m_2^\omega)^\omega (m_2^\omega m_1)^\omega &= (m_1 m_2^\omega)^\omega m_2^\omega (m_1 m_2^\omega)^\omega m_1 \\ &= (m_1 m_2^\omega)^\omega m_1 m_2^\omega m_2^\omega (m_1 m_2^\omega)^\omega m_1 \\ &= (m_1 m_2^\omega)^\omega m_1\end{aligned}$$

Since these are in fact both $\mathcal{R}$-equivalent idempotents, they are equal (Stiffler Jr, 1973). □

Theorem 14. C-RASP $\cap$ **REG** $=$ wpc(**Dy**). *Hence* $\mathbf{R} \subsetneq$ C-RASP $\cap$ **REG** $\subsetneq \mathbf{R}^\omega \subsetneq \mathbf{A} \subsetneq$ **REG**

[12] Choose $\rho_1 \in R_i$ just to produce the first arrow. Now we choose $\rho_2 \in M$ such that $\rho_1 s_0 \rho_2 = \lambda$, using that $\rho_1 s_0 \rho_2$ and λ both are in R_i. Now applying Lemma 59 (Claim 2) inductively to each j, there is an arrow $u_o t_0 \to_{(\rho_2, t')} u$.

Proof. The decision procedure shows that every finite monoid in C-RASP can be decomposed into wreath products of U_1 and $\mathcal{D}_k$ – in this case $U_1 \preceq \mathcal{D}_1$. In the other direction, $\mathcal{D}_k \in$ C-RASP, and thus wpc($\mathbf{Dy}$) contains only monoids in C-RASP, using appendix E.2). We use this to show the other strict inclusions.

- First, $\mathbf{R} = \text{wpc}(U_1)$. $U_1 \in \mathbf{Dy}$. Then we have that C-RASP $\cap$ **REG** $= \text{wpc}(\mathbf{Dy})$), so $\mathbf{R} \subseteq$ C-RASP $\cap$ **REG**. For a strict separation, $\mathcal{D}_1 \in$ C-RASP $\cap$ **REG** but $\mathcal{D}_1 \notin \mathbf{R}$ (Brzozowski & Fich, 1980).
- We have that C-RASP $\cap$ **REG** $= \text{wpc}(\mathbf{Dy})$. Then by proposition 7 we know $\mathcal{D}_k$ divides into a wreath product of U_1 and cyclic groups. We can move all U_1 factors to the left, obtaining a monoid in $\mathbf{R} \circ \mathbf{G}$ (Stiffler Jr, 1973). Furthermore, since all monoids C-RASP are aperiodic, this results in a monoid in $\mathbf{R}^\omega$, by theorem 13. For a strict separation, the monoid $M((ab+bba)^*) \in \mathbf{R}^\omega \setminus$ C-RASP (this can be computed by the decision procedure)
- First, $\mathbf{R}^\omega \subseteq \mathbf{A} \subsetneq$ **REG** by theorem 13. The strict separation can be witnessed by $M((ab+aabb)^*)$, which is aperiodic but has two idempotents in a single $\mathcal{R}$-class
- $\mathbf{A} \subsetneq$ **REG** is a standard fact. For instance, $M((b^*ab^*ab^*)^*)$ (i.e. $\mathbb{Z}/2\mathbb{Z}$) witnesses the strict separation.

□

J Automata proof of C-RASP $\cap$ REG

We attempt to construct a C-RASP recognizer by iterating over the reachability order. Whenever we encounter a nontrivial strongly connected component, we need to check if the recognizer can be extended to cover it. If we succeed for all strongly connected components, we have constructed a C-RASP recognizer. If we fail at some component, we have shown non-membership in C-RASP.

Definition 63. *We say that a* C-RASP *program $\mathcal{P}$ recognizes a language $\mathcal{L}$ if and only if there is a Boolean-valued predicate $P \in \mathcal{P}$ such that $[[P]](w\$, |w|+1) = \top \Leftrightarrow w \in \mathcal{L}$.*

Lemma 64 (Existence of Decision Algorithm)**.** *There is an algorithm that performs the following.*

Consider the finite alphabet Σ. Let \$ be a reserved EOS symbol, $\$ \notin \Sigma$.

Let $\mathcal{A} = \langle Q, \delta, q_0 \rangle$ be a DFA, where $\delta : Q \times \Sigma \to Q$. δ is canonically extended to $Q \times \Sigma^* \to Q$.

Let $W_1, \ldots, W_k \subset Q$ be the strongly connected components of the DFA, ordered such that, if W_i is reachable from W_j, then $i \geq j$. Let

$$U_i := \{w : \delta(q_0, w) \in \bigcup_{j \leq i} W_j\} \tag{11}$$

For $i = 1, \ldots, I_{max} \leq k$, the algorithm chooses an increasing sequence of C-RASP *programs $\mathcal{P}_1 \subset \mathcal{P}_2 \subset \ldots$ so that we can maintain the following invariant across i (trivially true at $i = 0$):*

For each $q \in Q$, there is a Boolean-valued predicate $P_{q,i} \in \mathcal{P}_i$ such that, for each $w \in U_i$ and each $\alpha \in Sigma$,

$$\delta(q_0, w) = q \Leftrightarrow [[P_q]](w\alpha, |w|+1) = \top \tag{12}$$

Furthermore, the algorithm terminates at $I_{max} = k$ if and only if all languages recognized by $\mathcal{A}$ are in C-RASP*. It terminates at $I_{max} < k$ if this is not the case.*

Proof. We first define the algorithm, and simultaneously inductively prove the claims about the algorithm. As a consequence of the invariant, at each step we note that one can define $Q_{q,i}$ such that, for each $w \in U_i$:

$$\delta(q_0, w\alpha) = q \Leftrightarrow [[Q_{q,i}]](w\alpha, |w|+1) = \top \tag{13}$$

by

$$Q_{q,i}(k) = \bigvee_{\alpha, q' : \delta(q', \alpha) = q} \Big[Q_\alpha(k) \wedge P_{q',i-1}(k) \Big] \tag{14}$$

We will use $Q_{q,i}$ without re-defining it. We define $\mathcal{P}_0 = \emptyset$. For notation, we index C-RASP formulas added to the program with a tuple such as $(i, \dots)$ or similar, where i indicates that the counter was added at step i. We perform induction over i.

Construction in the i-th step For any $w \in \Sigma^*$, we define the *entry state* $\mathfrak{e}_i(w)$ as the first state in W_i that is reached when running on w:

$$\mathfrak{e}_i(w) := \begin{cases} \delta(q_0, u\alpha) & \text{if } \delta(q_0, u\alpha) \in W_i, \delta(q_0, u) \notin W_i, \alpha \in \Sigma, w \in u\alpha\Sigma^* \\ \perp & \text{if no such } u \text{ exists} \end{cases} \tag{15}$$

We define

$$P_{i,0,0}(k) := \exists l \leq k : \bigvee_{q \in W_i} Q_{q,i-1}(l) \tag{16}$$

$$P_{i,0,0,q}(k) := \exists l \leq k : \left[Q_{q,i-1}(l) \wedge \neg \left[\exists m < l : P_{i,0,0}(m)\right]\right] \quad (\text{for } q \in W_i) \tag{17}$$

By definition, for $w \in \Sigma^*$,

$$[[P_{i,0,0,q}]](w, k) = \top \Leftrightarrow \mathfrak{e}_i(w) = q \tag{18}$$

Over the course of this iteration, we create maps Φ_j ($j = 1, 2, \dots$) assigning each edge $q_1 \to_\alpha q_2$ ($q_1 \in W_i$) paired with an entry state $q \in W_i$ to vectors in $\mathbb{Z}$ of some dimensionality d_j:

$$\Phi_j : W_i \times \Sigma \times W_i \to \mathbb{Z}^{d_j} \tag{19}$$

We will define these below. Each of these maps will be *balanced*, meaning that, whenever $q_1 \xrightarrow{\alpha_1} \dots \xrightarrow{\alpha_{k-1}} q_k \xrightarrow{\alpha_k} q_1$ is a path ($q_1 \in W_i$), then for all $q \in W_i$,

$$\sum_{l=1}^{k} \Phi_j(q_l, \alpha_l, q) = \mathbf{0} \in \mathbb{Z}^{d_j} \tag{20}$$

We define Φ_0 as $\Phi_0(q_l, \alpha, q) \equiv \langle\rangle \in \mathbb{Z}^0$ ($d_0 = 0$). Now $\phi : W_i \times \Sigma \times W_i \to \mathbb{Z}$ is *definable* from Φ_{j-1} if there is a map $\psi : \Sigma \times \mathbb{Z}^{d_{j-1}} \to \mathbb{Z}$ such that

$$\phi(q_l, \alpha_l, q) = \sum_{k=1}^{l} \psi\left(\alpha_k, \Phi_{j-1}(q_k, \alpha_k, q)\right) \tag{21}$$

for some path $q = q_1 \xrightarrow{\alpha_1} q_2 \to \dots \to q_l \xrightarrow{\alpha_l} q_{l+1}$ where $q_1, \dots, q_l \in W_i$. *In fact, (we don't need this at this point of the proof, but will need it later), when ϕ is balanced, this value does not depend on the chosen path by Lemma 68.*

At the j-th step, we consider the set $\mathcal{M}_j$ of balanced maps ϕ definable from Φ_{j-1}. This set is closed under addition and under multiplication with integers. Because the set of maps $\phi : W_i \times \Sigma \times W_i \to \mathbb{Z}$ is a finitely generated[13] module over the Noetherian ring $\mathbb{Z}$, the sub-module $\mathcal{M}_j$ has a finite integral basis, i.e., there is a finite subset $\mathcal{B}_j \subset \mathcal{M}_j$ such that each $\phi \in \mathcal{M}_j$ is an integer linear combination of elements of $\mathcal{B}_j$. We define Φ_j by appending all elements of $\mathcal{B}_j$ to Φ_{j-1}, so that each added dimension will correspond to a balanced map $\phi \in \mathcal{B}_j$.

We iterate $j = 1, 2, 3, \dots$ and terminate when all elements of $\mathcal{B}_j$ are in the span (over $\mathbb{Z}$) of the coordinates of Φ_{j-1}. Termination is assured because, as remarked above, the space of maps $\phi : W_i \times \Sigma \times W_i \to \mathbb{Z}$ is a finitely generated module over the Noetherian ring $\mathbb{Z}$.

We say that a pair of states $q_1, q_2 \in W_i$ is *separated* by Φ_j if their sets of outgoing transition labels under Φ_j are all distinct. That is, if for each entry state $q \in W_i$ we have:

$$\{\Phi_j(q_1, \alpha_1, q) : \alpha_1 \in \Sigma\} \cap \{\Phi_j(q_2, \alpha_2, q) : \alpha_2 \in \Sigma\} = \emptyset. \tag{22}$$

[13] Finitely generated because $|W_i|$ and $|\Sigma|$ are finite.

Let J be the j at which the algorithm terminates. We declare the algorithm "failed" if it terminates here without separating all state pairs. In this case, the automaton recognizes languages outside of C-RASP (Lemma 69). We declare it "successful" if it terminates separating all state pairs. In the latter case, we obtain the count-valued formulas (recall that $P_{i,0,0,q}$ checks if W_i was entered via q, (18)), for each $1 \leq t \leq d_J$:

$$[[P_{i,final,t}]](u\alpha, k) = \begin{cases} \Phi_J(\delta(q_0, u), \alpha, q)_t & \text{if } P_{i,0,0,q}(w, k) = \top \\ \text{(irrelevant)} & \text{if } P_{i,0,0,q}(w, k) = \bot, \forall q \end{cases} \tag{23}$$

from Lemma 67. We then add all Boolean-valued formulas

$$P_{i,final,t,s}(k) := \Big(P_{i,final,t}(k) = s\Big) \wedge P_{i,0,0}(k) \tag{24}$$

for all $s \in range(\Phi_J)$. Because Φ_J separates all edge pairs starting in the SCC W_i, we can derive Boolean-valued formulas $P_{q,i}$ satisfying (12).

□

Corollary 65. *The algorithm defined in the previous result runs in time $\mathcal{O}(poly(|Q|, |\Sigma|))$.*

Proof. The key here is that (i) the space of maps $\phi : W_i \times \Sigma \times W_i \to \mathbb{Z}$ forms a finitely generated $\mathbb{Z}$-module of dimension $|W_i|^2|\Sigma| \leq |Q|^2|\Sigma|$, (ii) the submodule of maps definable from some Φ_{j-1} has an explicit basis in terms of the indicator functions on equivalence classes of edges mapped to the same output by Φ_{j-1}, (iii) the balanced-ness condition can be expressed by a linear system of equations, (iv) $\mathcal{B}_j$ can be found in polynomial time.

Spelling out the algorithm:

1. Given Φ_{j-1}, we can obtain $\mathcal{B}_j$ as follows:
2. We first create the space W_j of functions ϕ definable from Φ_{j-1}
3. "Functions" can be thought of as vectors with integer entries over indices in $[|W_i||\Sigma||W_i|]$
4. To obtain W_j, it suffices to create a basis. This can consist of functions $\phi_{\alpha 0, label0}$ where
$$\phi_{\alpha 0, label0}(\alpha, label) := \begin{cases} 1 & \text{if } \alpha = \alpha 0 \wedge label = label0 \\ 0 & \text{otherwise} \end{cases} \tag{25}$$
constructed for each $\alpha \in \Sigma$ and each $label \in \mathbb{Z}^{d_{j-1}}$ produced by Φ_{j-1}
5. Given this basis, the 'balancedness' condition is then expressed by a homogenous linear system of equations, one for each simple cycle in the SCC. There will only be polynomially many equations. This is because any cycles which share an edge immediately have two states which cannot be separated (as both will have 0 as an outgoing label), and so we only need to consider cycles which do not share any edges.
6. Solving this linear system provides an integer basis for $\mathcal{B}_j$. It seems that the Smith Normal Form implemented in SymPy could be used for this.

□

Lemma 66. *Assume the algorithm finishes successfully. Then the* C-RASP *program constructed regognizes all languages recognized by $\mathcal{A}$.*

Proof. This is immediate: Take $\mathcal{L} = \{w : \delta(q_0, w) \in Q_{accept}\}$; then define

$$P_{\mathcal{L}}(k) := \bigvee_{q \in Q_{accept}} P_q(k) \tag{26}$$

$$w \in \mathcal{L} \Leftrightarrow [[P_{\mathcal{L}}]](w\$, |w|+1) = \top \tag{27}$$

□

Lemma 67. *For the t-th coordinate $\phi := (\Phi_j)_t$ ($t \leq d_j$), there is a count-valued* C-RASP *formula such that*

$$[[P_{i,j,t}]](u\alpha, k) = \begin{cases} \Phi_j(\delta(q_0, u), \alpha, \mathfrak{e}_i(w))_t & \text{if } P_{i,0,0,q}(w,k) = \top \\ \text{(irrelevant)} & \text{if } P_{i,0,0,q}(w,k) = \bot, \forall q \end{cases} \tag{28}$$

Proof. By induction over j. The claim is evident at $j = 0$. Now consider the inductive step. When $1 \leq t \leq d_{j-1}$, we take $P_{i,j,t} := P_{i,j-1,t}$, whose existence has been established previously by the inductive hypothesis. We now consider $d_{j-1} + 1 \leq t \leq d_j$. For a word w such that $[[P_{i,0,0}]](w,k) = \top$, let $\mathfrak{f}(w)$ be the first index f such that $\delta(q_0, w_{1:f}) \in W_i$ For the t-th coordinate ($d_{j-1} + 1 \leq t \leq d_j$), defined by a function ψ, we can define $P_{i,j,t}$ such that, when $w = u\alpha$, $k = |w|$:

$$\begin{aligned}
[[P_{i,j,t}]](w,k) &:= \sum_{s=1}^{k} \begin{cases} \psi(w_s, [[P_{i,j-1,1}]](w_{1:s}, s), \ldots, [[P_{i,j-1,d_{j-1}}]](w_{1:s}, s)) & \text{if } P_{i,0,0,q}(w,k) = \top \\ 0 & \text{if } P_{i,0,0,q}(w,k) = \bot, \forall q \end{cases} \\
&= \sum_{s=1}^{k} \begin{cases} \psi(w_s, \Phi_{j-1}(\delta(q_0, w_{1:s-1}), w_s, q) & \text{if } P_{i,0,0,q}(w,k) = \top \\ 0 & \text{if } P_{i,0,0,q}(w,k) = \bot, \forall q \end{cases} \\
&= \sum_{s=\mathfrak{f}(w)}^{k} \psi(w_s, \Phi_{j-1}(\delta(q_0, w_{1:s-1}), w_s, \mathfrak{e}(w))
\end{aligned}$$

Now consider the path $q_1 \xrightarrow{\alpha_1} q_2 \to \cdots \to q_k \xrightarrow{\alpha_k} q_{k+1}$ where $\alpha_i = w_{\mathfrak{f}(w)+i}$. Hence, by (21) and Lemma 68,

$$[[P_{i,j,t}]](w,k) = \phi(\delta(q_0, w_{1:s} - 1), w_s, \mathfrak{e}(w)) \tag{29}$$

□

Lemma 68. *If $\phi : W_i \times \Sigma \times W_i \to \mathbb{Z}$ is balanced, then its value does not depend on the choice of the path used in (21).*

Proof. Assume otherwise. Then there would be words w, w' such that $\delta(q_0, w) = \delta(q_0, w') = q$ and also where the minimal prefixes reaching W_i evaluated to the same state q' – yet with $\phi(w) \neq \phi(w')$. Then there are two paths from q' to q evaluating to different values under ϕ. This is a contradiction. □

Lemma 69. *If failure is declared at some step I_{max}, then $\mathcal{A}$ recognizes some language that is not recognized by any* C-RASP *program.*

Proof. Assume all languages recognized by $\mathcal{A}$ are in C-RASP. Let i be the iteration where the algorithm failed. We consider the languages

$$\mathcal{L}_{q,q'} := \{w\alpha : \delta(q, w) \in W_i, \delta(q, w\alpha) = q'\} \tag{30}$$

for each $q \in W_i$, $q' \in Q$. We note that if all languages recognized by $\mathcal{A}$ are recognized in C-RASP, then so is each $\mathcal{L}_{q,q'}$.

To obtain a contradiction, we now show that at least one of these languages cannot be recognized in C-RASP.

Let B be the maximum word attained by any coordinate of Φ_i. Let $\mathcal{S}(\Phi_J)$ be the set of strings over Σ with the values in each coordinates (truncated to B) attached to each symbol, forming words over an extended alphabet Σ_Φ. Now consider each such language $\mathcal{L}_{q,q'}$ as a subset of $\mathcal{S}(\Phi_J)$.

As the algorithm terminated, all balanced maps ϕ are linear combinations of dimensions of Φ_J; hence, any balanced map can be directly determined from the relabeled symbols.

Now we consider a C-RASP formula for the language $\mathcal{L}_{q,q'} \subseteq \mathcal{S}(\Phi_J)$ over the relabeled alphabet; we can choose the formula to have minimal depth.

First, we note that, for at least one $\mathcal{L}_{q,q'}$, this formula must have depth at least one, because Φ_J does not separate all pairs of states.

Consider the depth-one formulas, which are Boolean combinations of linear inequalities over counts in Σ_Φ:

$$\sum_{\alpha \in \Sigma_\Phi} \lambda_\alpha \# Q_\alpha \leq \kappa \tag{31}$$

For each such term $\sum_{\alpha \in \Sigma} \lambda_\alpha$, we distinguish two cases:

The first case is that the term sums to zero on cycles. The second case is that the term does not sum to zero on cycles.

In the first case, because the procedure has already terminated, the information can actually be directly read off from the label in the extended alphabet, and be replaced with a Boolean combination of depth-zero formulas (if not, we would have chosen this as new annotation and done another iteration). We can thus transform the formula to a form where the depth-one formulas only use inequalities over counts that do not sum to zero on cycles.

Now, we may prepend, without changing membership in $\mathcal{L}_{q,q'}$, very long iterated cycles $q \to q$, in such a way as to make the depth-one formulas constant on bounded-length suffixes to a very large power of the loop. We can then obtain a C-RASP formula for recognizing $\mathcal{L}_{q,q'}$ while reducing the depth by one. This leads to a contradiction, as we had assumed the depth of the formula to be minimal.

□

K Implementation

A sketch of the algorithm is given below, which is close to the python implementation we have provided For each simple loop in a strongly connected component, the helper function BALANCEDLABELSBASIS computes the basis of all morphisms into $\mathbb{Z}$ given the constraint that all loops must be "balanced" (i.e. have the images of the symbols sum to 0). By computing the nullspace of this basis, we find all possible morphisms that sum to 0 on loops, and iteratively relable the symbols according to these morphisms. At the end, the helper function SEPARATED checks if all states have differing sets of outgoing transition labels (hence the states can be distinguished only using iterated counting – and thus being expressible in C-RASP).

Algorithm 1 C-RASP Membership

```
1: procedure DecideMembership(𝒜)
2:     SCCs ← StronglyConnectedComponents(𝒜)
3:     for all SCC ∈ SCCs do
4:         B_prev ← ∅
5:         B_curr ← BalancedLabelsBasis(SCC)
6:         if Nullspace(B_cur) ≠ ∅ then
7:             while B_curr ∉ Span(B_prev) do
8:                 labels[σ_i] ← [v_i]_{v∈B_curr}
9:                 for all loop ∈ LoopsAround(q_0, SCC) do
10:                    for all 0 ≤ i ≤ Len(loop) do
11:                        (q_i, σ_i, q_{i+1}) ← loop[i]
12:                        SCC.δ.Replace((q_i, σ, q_{i+1}), (q_i, [σ_i, Σ_{1≤j≤i} labels[σ_j]], q_{i+1}))
13:                    end for
14:                    B_prev ← B_curr
15:                    B_curr ← BalancedLabelsBasis(SCC)
16:                end for
17:            end while
18:        end if
19:        if ¬Separated(SCC) then
20:            return ⊥
21:        end if
22:    end for
23:    return ⊤
24: end procedure
```